Mid-Semester Question Bank — Answers

CSAI2017P: Applied Machine Learning

Units I–IV and Unit V (part) • L1–L10 • 314 questions

MARKING SCHEME — NOT FOR CIRCULATION

Instructor: Dr. Tofik Ali **Assignment due:**

How to use this

These are the answers to all 314 questions in the bank, with the working for every numerical one.

Work the question first. Reading a solution produces the feeling of understanding and almost none of the substance — that is why the bank told you to do the numericals with a pen, and it is still true now that the answers are in your hands.

Numerical values are exact to the digits shown. If you rounded intermediate steps you may differ in the last decimal or two; that is rounding, not an error. If you differ by more than that, the working is here so you can find where the two of you parted company.

Theory answers say what has to be present, not words to match. Your wording will differ from the wording here, and that is expected. What matters is whether the named ideas are in your answer.

Statements to judge carry a verdict and the reason behind it. The reason is the part that does the work — a verdict on its own does not answer the question that was asked.

Several questions have more than one good route, and a route different from the printed one is not automatically worse. If you reached a different answer and can show your reasoning, bring it to office hours.

If you change a question's numbers, change them in the script first and paste the new output here. A key that has drifted from its source is worse than no key.

Unit I — Introduction and the Python ML stack

§A — Two-mark questions

U1.1 "A program learns from experience E with respect to a class of tasks T and performance measure P, if its performance at T, as measured by P, improves with E." For pass prediction: T = classify a student as pass/fail; E = records of past students with their outcomes; P = accuracy or recall on students not in that record.

U1.2 (a) Supervised regression — signal is the recorded rainfall for each past month. (b) Unsupervised — no target; the signal is structure in the feature space. (c) Reinforcement — a delayed scalar reward (the score), not a per-step label. *0.5 each classification, 0.5 for naming signals.*

U1.3 AI $\supset$ ML $\supset$ DL. Not right for deep learning: a 500-row tabular credit dataset — a gradient-boosted tree usually matches or beats a neural network and trains in seconds.

U1.4 Any three of: the rule is already known and writable; there are no examples of the answer; the cost of an error is unbounded and the model cannot explain itself; the data distribution at deployment differs from training; a simple baseline already meets the requirement.

U1.5 NumPy — n-dimensional numeric arrays and vectorised arithmetic. Pandas — labelled heterogeneous tables, I/O, cleaning. scikit-learn — estimators, transformers, model selection.

U1.6 `fit` learns parameters from data and stores them; `transform` applies stored parameters; `predict` applies a learned model. `fit_transform` on test data would recompute the parameters *from* the test data — the definition of leakage. *The last sentence is the mark.*

U1.7 `head` (what the rows look like), `info` (dtypes and non-null counts), `describe` (location, spread, range per column), `isna().sum()` (where the holes are).

U1.8 "The axis I name is the one that disappears." $(3, 4)$: `sum(axis=0)` $\to (4,)$; `sum(axis=1)` $\to (3,)$.

U1.9 Training error low and test error high. The diagnostic is the *gap* between them, not either number alone.

U1.10 Use, in training or model selection, of information not available at prediction time — in practice, information from the evaluation data. Two examples from: scaler/imputer fitted on everything; feature selection on everything; resampling before the split; target encoding over all rows.

U1.11 Once you compare models on the test set, you have selected using it, and the winner's score is optimistically biased by the selection. The validation set absorbs the selection; the test set is looked at once, at the end.

U1.12 Two decimal places on a score from 38 test rows implies a precision the sample cannot support — one row is worth 2.6 accuracy points. Report "95%", or better, a cross-validated mean with its spread.

U1.13 Need scaling: k-NN, RBF-kernel SVM, K-means, and any gradient-trained linear model (distance or step size depends on units). Do not need it: decision trees and random forests — a split asks "is $x_j \le t$?", and a monotone rescaling $x \mapsto (x - a)/b$ with $b > 0$ moves t to $(t - a)/b$ and sends exactly the same rows left.

§B — Five-mark questions

U1.14 Verified.

Expected answer

(a) Distances from $(4.6, 1.4)$:

point	distance	species
$(1.5, 0.3)$	**3.2894**	setosa
$(1.4, 0.2)$	**3.4176**	setosa
$(4.8, 1.5)$	**0.2236**	versicolor
$(4.2, 1.2)$	**0.4472**	versicolor
$(5.9, 2.4)$	**1.6401**	virginica

Working for the third: $\sqrt{0.2^2 + 0.1^2} = \sqrt{0.05} = 0.2236$.
(b) 1-NN $\to$ **versicolor** (0.2236 is smallest).
(c) Three nearest are versicolor, versicolor, virginica $\to$ vote 2–1 $\to$ **versicolor**.

(d) In millimetres, petal width is multiplied by 10 while length stays in cm. Width differences then dominate the squared distance — the metric is no longer measuring what it was. Nothing about the flowers changed; only the units did. This is the whole case for standardising before any distance-based method.

U1.15 Verified.

Expected answer

(a) $e = (0.5, -0.5, 1.0, -1.0)$. MSE $= (0.25 + 0.25 + 1 + 1)/4 =$ **0.625**; MAE $= 3/4 =$ **0.75**; RMSE $= \sqrt{0.625} =$ **0.7906**.
(b) $\hat{y}_3 = 1.0 \Rightarrow e = (0.5, -0.5, 7.0, -1.0)$. MSE $= (0.25 + 0.25 + 49 + 1)/4 =$ **12.625**; MAE $= 9/4 =$ **2.25**; RMSE $= \sqrt{12.625} =$ **3.5532**.
(c) MSE grew ×20.2; MAE ×3.0; RMSE ×4.49. The error went from 1 to 7, a factor of 7; MAE is linear so it grew by roughly that factor spread over four points, MSE is quadratic so it grew by roughly 7^2 spread over four, and RMSE — being the square root of MSE — sits between them and is back in the units of y.

U1.16 Verified. (a) $0.2 \times 569 =$ **114** test, **455** train. (b) 10-fold on 569: validation ≈ **57** rows, training ≈ **512**. (c) **10** fits. (d) $n(k-1) = 569 \times 9 =$ **5121** row-fits. *1.25 each.*

U1.17 (a) **Degree 3** — lowest test RMSE (1.10), which is the only column that estimates future performance. (b) Degree 12 has driven training error to 0.57 by fitting the noise; test error explodes to 40.52, so the fitted curve is wild between the training points. (c) Training error can always be driven to zero by adding capacity — it measures memorisation, not generalisation, so it cannot arbitrate between models. (d) Training curve falls monotonically; test curve is U-shaped with its minimum at degree 3; everything to the right of that minimum is the overfitting region.

U1.18 (a) **0.50** — the labels are coin flips and the features are noise, so no procedure can beat chance in expectation. (b) The selector computed its F-statistics using every row's target, including rows that later became validation folds. Among 5000 pure-noise columns, roughly 20 will correlate with y across the whole sample by chance alone; the selector finds exactly those, and the correlation it exploited is still present in the validation rows because those rows helped choose the columns. The classifier then "confirms" a pattern that the selection step planted. (c) Move `SelectKBest` inside a `Pipeline` with the classifier and cross-validate the pipeline.

U1.19 Expect: `load_*`; `train_test_split(test_size=0.25, random_state=0, stratify=y)`; `make_pipeline(StandardScaler(), KNeighborsClassifier(5))`; `.fit(X_train, y_train)`; `.score(X_test, y_test)`. The two leakage-preventing lines are `stratify=y`/`random_state` (an honest, reproducible split) and the pipeline construction (the scaler is fitted on training folds only).

U1.20 A Python loop executes interpreted bytecode and boxes every element as a Python object; NumPy dispatches once into a compiled C loop over a contiguous typed buffer, with no per-element interpreter overhead and with SIMD available. The loop wins when the operation is not expressible elementwise, when it must exit early, or when the array is small enough that NumPy's call overhead dominates.

§C — Ten-mark questions

A complete §C answer covers the listed points, arrives somewhere rather than trailing off, and carries at least one concrete numerical illustration. An answer with no numbers anywhere caps at 6 of 10 regardless of prose quality.

U1.21 Must cover: training score measures memorisation and can be driven to zero (the degree-12 row); a held-out set estimates performance on unseen rows; CV averages over k disjoint held-out sets and so has lower variance than one split; three-way split — train fits, validation selects, test estimates once; two bias mechanisms, e.g. selecting a model on the test set, and any preprocessing step fitted before the split. Numerical illustration: the overfitting table, or the 0.81-on-noise leak.

U1.22 Expected chain: inspect (`head/info/describe/isna`) → split with `stratify` and a seed → `ColumnTransformer` with imputers, scaler, encoder → model inside a `Pipeline` → `StratifiedKFold` cross-validation on the training portion → tune on validation or by nested CV → one look at the test set → report a mean with its spread. At each stage the failure: unseen categories (`handle_unknown`), `NaN` reaching the scaler, statistics computed over test rows, selecting on the test set. The pipeline prevents the middle three by construction.

U1.23 Rule-based: written by hand, adapts only when a human edits it, debugged by reading, fully explainable, degrades predictably. Learned: induced from labelled examples, adapts by retraining, debugged by examining data and errors, explainable only with effort, can fail silently under distribution shift. Choose rules when the rule is known, stable, and must be auditable; choose learning when you have examples of the answer but no statement of the rule.

U1.24 The uniform API means any estimator can be dropped into any pipeline, any cross-validator, any grid search, without those tools knowing what the estimator is. Concretely, it makes the leakage bug impossible: because a `Pipeline` is itself an estimator with `fit`/`predict`, `cross_val_score` refits the *whole* chain inside each fold, so a scaler cannot accidentally be fitted once outside the loop.

Unit II — Loss functions

§A — Two-mark questions

U2.1 MSE $= \frac{1}{n}\sum r_i^2$, MAE $= \frac{1}{n}\sum |r_i|$, RMSE $= \sqrt{\text{MSE}}$. MSE is most sensitive: doubling a residual quadruples its contribution; equivalently $\partial\text{MSE}/\partial\hat{y}$ grows linearly with the residual while MAE's is constant ± 1.

U2.2 *Loss* — the penalty on one example. *Cost/objective* — the aggregate the optimiser minimises. *Metric* — the number reported to a human, which need not be differentiable. Example: train with cross-entropy, report F_1; train with Huber, report RMSE.

U2.3 It must be minimised by a correct prediction, and it must be differentiable (or subdifferentiable) so a gradient method can move on it.

U2.4 Squared → the **mean**. Absolute → the **median**. 0–1 → the **mode**.

U2.5 $L_\delta(r) = \frac{1}{2}r^2$ for $|r| \le \delta$, else $\delta(|r| - \frac{1}{2}\delta)$. The second branch is the tangent to the

first at $r = \delta$: written that way the value *and* the slope match at the join, so the loss is differentiable everywhere. Plain $\delta|r|$ would match neither.

U2.6 δ is in the units of y. Wrong way: copying a value from a tutorial. Right: cross-validation, or a robust estimate of the residual scale.

U2.7 It is piecewise constant — zero gradient almost everywhere, so nothing to descend — and it takes very few distinct values on a small batch, so it cannot distinguish a barely-right model from a confidently-right one.

U2.8 BCE $= -\frac{1}{n}\sum_i [y_i \log p_i + (1 - y_i)\log(1 - p_i)]$. For a single example it is $-\log$(probability given to the correct class).

U2.9 Categorical expects one-hot labels, shape (n, K); sparse expects integer indices, shape $(n,)$. They give **identical** numbers — 2-D means categorical, 1-D means sparse.

U2.10 Memory and speed. $K = 10{,}000$ with a batch of 256 means a one-hot matrix of 2.56 million numbers of which 256 are non-zero; the sparse form indexes directly and never builds it.

U2.11 $\log 0$ is undefined and $-\log$ of a value near zero overflows to ∞, which propagates `nan` through the gradients. Clipping bounds the loss without materially changing it.

U2.12 $m_i = y_i f(x_i)$ — positive when correct, and its magnitude is the confidence. Hinge $= \max(0, 1 - m_i)$.

U2.13 Hinge becomes *exactly* zero once $m \geq 1$ and stays there; logistic loss approaches zero but never arrives. Consequence: only points with $m < 1$ contribute gradient, so an SVM's solution is determined by a handful of points — the support vectors.

U2.14 $L = \max(0,\ d(a,p) - d(a,n) + \alpha)$ on an anchor, a positive and a negative. α is the margin: how much closer the positive must be than the negative before the triplet costs nothing.

U2.15 Most random triplets already satisfy the margin, giving zero loss and zero gradient, so nearly all the compute is wasted. Remedy: **triplet mining** — selecting hard or semi-hard triplets.

U2.16 A differentiable, tractable loss that upper-bounds the loss you actually care about. The two standard surrogates for 0–1 are cross-entropy (logistic) and hinge.

U2.17 (a) Huber, or MAE. (b) Sparse categorical cross-entropy. (c) Triplet loss. (d) Binary cross-entropy. *0.5 each.*

§B — Five-mark questions

U2.18 Verified.

Expected answer

(a) $r = y - \hat{y} = (-0.5,\ 1.0,\ -2.5,\ 1.0,\ 0)$; $r^2 = (0.25,\ 1,\ 6.25,\ 1,\ 0)$, $\sum = 8.5$; $|r| = (0.5,\ 1,\ 2.5,\ 1,\ 0)$, $\sum = 5$.
(b) MSE $= 8.5/5 =$ **1.7**; MAE $= 5/5 =$ **1.0**; RMSE $= \sqrt{1.7} =$ **1.3038**.

(c) Worst point $r = -2.5$: share of SSE $= 6.25/8.5 =$ **0.7353**; share of SAE $= 2.5/5 =$ **0.5000**.
(d) One point in five carries 74% of the squared error but only 50% of the absolute error. Under MSE that single row effectively decides the fit; under MAE it is one voice among five. This is the whole difference between the two losses, in two numbers.

U2.19 Verified. Residuals as above.

Expected answer

δ	per-point loss	quadratic branch	mean
1.0	0.125, 0.5, 2.0, 0.5, 0	all but $r = -2.5$	**0.625**
1.5	0.125, 0.5, 2.625, 0.5, 0	all but $r = -2.5$	**0.75**
3.0	0.125, 0.5, 3.125, 0.5, 0	*all five*	**0.85**

Check at $\delta = 1.5$: $1.5 \times (2.5 - 0.75) = 2.625$. At $\delta = 3$ every $|r| \le 3$, so every point takes $\frac{1}{2}r^2$ and the mean is $8.5/(2 \times 5) = 0.85$ — exactly half the MSE. As $\delta \to \infty$ the Huber loss becomes $\frac{1}{2}\times$ MSE: no point ever reaches the linear branch, and all robustness is lost. As $\delta \to 0$ it becomes $\delta\times$ MAE.

U2.20 Verified.

Expected answer

(a) Clean: mean $= 146/6 =$ **24.3333**, median $= (24 + 25)/2 =$ **24.5**.
(b) With 148: mean $= 266/6 =$ **44.3333**, median $=$ **24.5** — unmoved.
(c) Squared error is minimised by the mean, **44.3333**; absolute error by the median. A grid search over $[0, 200]$ returns **24.0** for the absolute error rather than 24.5, because with an even n the absolute loss is *flat* on the whole interval $[24, 25]$ and `argmin` returns the left end; every point in that interval is a minimiser.
(d) The constant a loss chooses *is* what the model becomes when it has no features to work with. A model trained on MSE is a model of the average case and inherits the mean's zero breakdown point; a model trained on MAE is a model of the typical case. The one corrupted mark moved the MSE answer by 20 marks and the MAE answer by nothing.

U2.21 Verified.

Expected answer

(a) Probability on the correct class: (0.8, 0.7, 0.6, 0.9).
(b) $-\log$: (0.2231, 0.3567, 0.5108, 0.1054); sum 1.1960; BCE = **0.2990**.
(c) With $p_3 = 0.02$: $-\log 0.02 = 3.9120$, so the sum becomes $0.2231 + 0.3567 + 3.9120 + 0.1054 = 4.5972$ and BCE = **1.1493** — an increase of **284**%.
(d) The loss is a negative logarithm, which is unbounded as $p \to 0$. A prediction of 0.01 on the correct class costs 4.61 against 0.01 for a prediction of 0.99 — a factor of **458**. One confidently wrong row can therefore outweigh several hundred quietly correct ones.

U2.22 Verified.

Expected answer

(a) Correct-class probabilities are the diagonal: 0.6, 0.7, 0.6. $-\log$: 0.5108, 0.3567, 0.5108; mean = **0.459442**.
(b) Identical: **0.459442**.
(c) They must agree — one-hot labels zero out every term except the correct class, which is exactly what integer indexing selects. Same loss, two label representations.
(d) One-hot for $K = 10{,}000$ and a batch of 256 is 2,560,000 numbers of which **256** are non-zero; the sparse form stores 256 integers. Roughly a 10,000× saving on the label tensor.

U2.23 Verified.

Expected answer

p	$-\log p$	ratio to $p = 0.99$
0.99	0.0101	1.0
0.90	0.1054	10.5
0.50	**0.6931**	69.0
0.10	2.3026	229.1
0.01	4.6052	**458.2**

The $p = 0.50$ row is exactly $\log 2 = 0.6931$ — the loss of a model that has learnt nothing and says so honestly. It is the natural baseline for binary cross-entropy, and a training run whose loss sits at 0.693 has not started.

U2.24 Verified.

Expected answer

(a) $m = yf = (1.5,\ 0.4,\ 0.2,\ 0.9)$ — all positive.
(b) Hinge $= \max(0, 1 - m) = (0,\ 0.6,\ 0.8,\ 0.1)$; mean $= 1.5/4 =$ **0.375**.
(c) Squared hinge $= (0,\ 0.36,\ 0.64,\ 0.01)$; mean $= 1.01/4 =$ **0.2525**.
(d) Every margin is positive, so every point is on the correct side of the boundary — but three of them sit inside the margin band $m < 1$. Hinge loss does not ask for correctness; it asks for correctness *by a stated margin*. Only the first point, at $m = 1.5$, has bought enough room to stop contributing. This is why an SVM's solution depends on so few points: everything with $m \geq 1$ has zero loss and zero gradient and drops out of the problem.

U2.25 Verified. (a) $\max(0, 0.9 - 1.6 + 1.0) =$ **0.3** — non-zero, so there is a gradient. (b) $\max(0, 0.4 - 2.1 + 1.0) = \max(0, -0.7) =$ **0** — the margin is already satisfied, no gradient, the triplet teaches nothing. (c) $\max(0, 1.2 - 1.4 + 0.5) =$ **0.3** — non-zero even though the ordering is correct, because the gap 0.2 is below $\alpha = 0.5$. Case (b) is the common case under random sampling, which is why almost all compute is wasted without mining. *1 each + 2 for the sampling conclusion.*

U2.26 The two numbers are not comparable: MAE and RMSE are different functions of the same residuals, and MAE $\leq$ RMSE always. On the L2 worked example, identical predictions gave MAE 1.0 and MSE 1.375 — neither model was better; only the ruler changed. To compare the two models, evaluate both under *one* metric on *one* held-out set, chosen for what the application cares about.

U2.27 $\partial\text{MSE}/\partial r = 2r$: continuous, and it shrinks as the fit improves, so steps get gentler near the optimum. $\partial\text{MAE}/\partial r = \text{sign}(r)$: constant magnitude, so the step size never eases off, and it is undefined at $r = 0$ — the optimiser oscillates about the minimum. On an even-sized sample the absolute loss is flat across the whole interval between the two middle values, so the gradient is exactly zero on a set of minimisers and the answer returned depends on the solver.

U2.28 (a) BCE on a softmax treats the K outputs as K independent binary problems; the softmax constraint $\sum_k p_k = 1$ makes them dependent, so the loss is mis-specified and the gradients fight each other. (b) Categorical CE expects shape $(n, 4)$; integer labels of shape $(n,)$ either raise a shape error or — worse — broadcast silently and compute nonsense. (c) Four sigmoids do not sum to 1, so "probabilities" can total 3.2; categorical CE divides by nothing and the loss is no longer a proper scoring rule.

U2.29 All three surrogates upper-bound the 0–1 step, which is what makes minimising them a sensible proxy. Hinge reaches *exactly* zero at $m = 1$ and is flat thereafter; logistic approaches zero asymptotically and never arrives, so it keeps pushing confident points further. For very negative margins hinge grows linearly and squared hinge quadratically — so squared hinge is far more sensitive to a badly misclassified point, in the same way MSE is more sensitive than MAE.

§C — Ten-mark questions

U2.30 Required coverage: the three constant minimisers (mean / median / mode) with the argument; a worked contaminated sample showing the mean move and the median hold (U2.20 numbers are fine); Huber as the piecewise compromise, δ in units of y, chosen by CV; a loss/metric divergence example. The thesis to land: the loss determines what the model *becomes*, not merely how it is scored.

U2.31 Coverage: BCE, CCE and sparse CCE with formulae, label formats and output activations; the identity between the last two; $-\log$ of the correct class as the unifying form; the confidently-wrong table with the 458× figure and $\log 2 = 0.6931$ as the honest-ignorance baseline; why accuracy cannot serve — piecewise constant, few distinct values, no gradient.

U2.32 A comparison table plus a worked five-point example. Expect: MSE quadratic / outlier-sensitive / fits the mean / smooth / gradient $2r$ / no hyperparameter / clean symmetric data. MAE linear / robust / fits the median / kink at 0 / gradient ± 1 / none / untrusted outliers. Huber piecewise / bounded influence / between the two / smooth everywhere / one hyperparameter δ in units of y / unavoidable outliers.

U2.33 Coverage: $m = yf$; hinge geometry and the margin band; support vectors as the points with $m < 1$; squared hinge and its quadratic tail; triplet loss for metric learning; mining because random triplets give zero gradient.

§D — Statements

U2.34 **False.** They are different functions of the same residuals and MAE $\leq$ RMSE by construction; the smaller number reflects the ruler, not the model.

U2.35 **True.** The linear branch is the tangent to the quadratic at $r = \delta$, so value *and* first derivative agree at the join.

U2.36 **False.** Same loss, different label encoding; they return identical values (U2.22: both 0.459442). Only memory differs.

U2.37 **False.** Hinge is zero only for $m \geq 1$. U2.24 has four correctly classified points and three of them cost something.

U2.38 **False on the second clause.** RMSE is a monotone function of MSE, so on *one* dataset the ranking is identical — but the claim as written is loose: RMSE is in the units of y while MSE is in units of y^2, which matters whenever the two are compared across differently scaled targets or combined with another term.

Unit III — Optimiser functions

§A — Two-mark questions

U3.1 $w \leftarrow w - \eta \nabla J(w)$, where w is the parameter vector, $\eta > 0$ the learning rate and $\nabla J = [\partial J/\partial w_1, \ldots, \partial J/\partial w_p]^\top$.

U3.2 Any two: the closed form costs $O(np^2 + p^3)$ and is infeasible for large p; $X^\top X$ may be singular or badly conditioned; most losses of interest have no closed form at all; gradient methods stream over data that does not fit in memory.

U3.3 An *epoch* is one complete pass over the training set; an *update* is one application of the update rule. Batch: 1 update/epoch. SGD: n. Mini-batch: n/B.

U3.4 $0 < \eta < 2/L$. At $\eta = 1/L$ the multiplier $|1 - \eta f''|$ is exactly zero, so a quadratic is solved in one step — that step is **Newton's method**.

U3.5 Pick i at random; $w \leftarrow w - \eta \nabla \ell_i(w)$. It works because the per-example gradient is *unbiased*: $\mathbb{E}_i[\nabla \ell_i(w)] = \nabla J(w)$.

U3.6 $\sum_t \eta_t = \infty$ (able to travel any distance) and $\sum_t \eta_t^2 < \infty$ (noise eventually suppressed). $\eta_t = \eta_0/(1 + ct)$ satisfies both.

U3.7 It falls as $1/\sqrt{B}$. So quality improves as $\sqrt{B}$ while cost grows as B — doubling B buys 29% less noise for 100% more work.

U3.8 $v \leftarrow \beta v + \nabla J(w)$; $w \leftarrow w - \eta v$. β sets how much accumulated velocity is retained — the memory of past gradients. As $\beta \to 1$ the effective step $\eta/(1 - \beta) \to \infty$: oscillation across a ravine is damped but the iterate overshoots badly.

U3.9 $\eta_{\text{eff}} \approx \eta/(1 - \beta)$. Going from 0.9 to 0.99 multiplies the effective step by **10**, so η must be divided by 10 to keep the same behaviour.

U3.10 $\kappa = \lambda_{\max}/\lambda_{\min}$ of the Hessian. GD converges at rate $(\kappa - 1)/(\kappa + 1) \approx 1 - 2/\kappa$; momentum improves this to $\approx 1 - 2/\sqrt{\kappa}$.

U3.11 On a well-conditioned problem ($\kappa \approx 1$) there is no oscillation to damp, so the retained velocity only causes overshoot. On the 1-D quadratic, $\beta = 0.9$ was measured 2.5× *slower* than $\beta = 0$.

U3.12 $s_t = s_{t-1} + g_t \odot g_t$; $w \leftarrow w - \frac{\eta}{\sqrt{s_t + \varepsilon}} \odot g_t$. s_t accumulates the *sum of squared gradients*, per coordinate.

U3.13 The sum only ever grows, so the effective learning rate $\eta/\sqrt{s_t} \approx \eta/(\bar{g}\sqrt{t}) \propto 1/\sqrt{t}$ decays without bound and training stalls before convergence on long runs.

U3.14 $s_t = \gamma s_{t-1} + (1-\gamma) g_t \odot g_t$; same second line. The sum becomes an **exponentially weighted moving average**, so old gradients decay out and the denominator can shrink again.

U3.15 Adagrad's convergence guarantee. Its $1/\sqrt{t}$ decay *is* a Robbins–Monro schedule on a convex problem; RMSprop's average never settles, so it tracks a moving target but has no such proof.

U3.16 That the update should have the same units as the parameter it updates. It removes η entirely, replacing it with the RMS of recent updates.

U3.17 With $d_0 = 0$ the first numerator is $\sqrt{\varepsilon}$, so the first step is proportional to $\sqrt{\varepsilon}$ and the whole trajectory is set by it. A hyperparameter has been disguised as a stability constant.

U3.18 $m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$; $v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$; $\hat{m}_t = m_t/(1-\beta_1^t)$, $\hat{v}_t = v_t/(1-\beta_2^t)$; $w \leftarrow w - \eta \hat{m}_t/(\sqrt{\hat{v}_t} + \varepsilon)$.

U3.19 $m_0 = v_0 = 0$, so for a constant gradient $m_t = (1-\beta_1^t) g$ — both moments are shrunk towards zero early on, and by *different* factors because $\beta_1 \neq \beta_2$. It is the mismatch, not the shrinkage, that breaks the ratio.

U3.20 At $t = 1$: $\hat{m}_1 = (1-\beta_1) g_1/(1-\beta_1) = g_1$ and $\hat{v}_1 = (1-\beta_2) g_1^2/(1-\beta_2) = g_1^2$. So $\hat{m}_1/\sqrt{\hat{v}_1} = g_1/|g_1| = \pm 1$ and the step is exactly $\pm\eta$.

U3.21 Not a better final answer — on a well-conditioned convex problem with a tuned η the optimiser is worth about one per cent. What they buy is a **wider window of learning rates that work**: Adagrad worked over 2.24 decades of η against SGD's 1.31.

U3.22 **Curvature** (a ravine: coordinates want different step sizes because of differing λ_i) and **frequency** (sparsity: a rare feature is updated rarely). Momentum addresses the first and does nothing for the second.

U3.23 Still falling steeply → η too small, multiply by 3. Falls fast then flattens → healthy. Falls then bounces → η slightly too large, or the SGD noise floor. Rises or `nan` → past the stability threshold, divide η by 10.

U3.24 That it is genuinely $B = 1$ — true stochastic gradient descent, not the mini-batch method that the name "SGD" usually denotes elsewhere.

U3.25 $v \leftarrow \beta v + g$ (PyTorch, D2L) versus $v \leftarrow \beta v + (1-\beta) g$ (Ng, the Adam paper). They differ by a factor of $1-\beta$, so an η tuned under one is wrong by $10\times$ under the other at $\beta = 0.9$.

§B — Five-mark questions

U3.26 Verified.

Expected answer

(a) $f'(w) = 2(w-4)$, so $w \leftarrow w - 0.15 \cdot 2(w-4) = 0.7w + 1.2$.
(b)

k	w_k	$f(w_k)$	$f'(w_k)$
0	0.0000	16.0000	−8.0000
1	1.2000	7.8400	−5.6000
2	2.0400	3.8416	−3.9200
3	2.6280	1.8824	−2.7440

$w_4 = \mathbf{3.0396}$.
(c) Gradients fall in the fixed ratio $5.6/8 = 3.92/5.6 = \mathbf{0.7}$, and $|1 - 2\eta| = |1 - 0.30| = 0.7$. Convergence is *geometric*: every step buys the same percentage of the remaining distance.
(d) $w_k = 4 - 4(0.7)^k$; $w_4 = 4 - 4(0.2401) = \mathbf{3.0396}$. Agrees.

U3.27 Verified. (a) $f'' = 6$, so $0 < \eta < 2/6 = \mathbf{0.3333}$. (b) $\eta = 1/6 = \mathbf{0.1667}$. (c) Multiplier $|1 - 0.4 \times 6| = 1.4 > 1$: $w_1 = \mathbf{-1.4}$, $w_2 = \mathbf{1.96}$, $w_3 = \mathbf{-2.744}$, $w_4 = \mathbf{3.8416}$ — oscillating with growing magnitude, i.e. diverging. (d) The loss curve rises, then produces `inf` or `nan`. First action: divide η by 10. A diverging loss is almost always a learning-rate problem before it is anything else.

U3.28 Verified.

Expected answer

(a) $\nabla J(w) = -\frac{2}{3}\sum x_i(y_i - wx_i)$. At $w_0 = 0$: $\sum x_i y_i = 3 + 10 + 36 = 49$, so $g = -\frac{2}{3}(49) = \mathbf{-32.6667}$ and $w = 0 + 0.05(32.6667) = \mathbf{1.6333}$.
(b) SGD, per-example gradient $2x_i(wx_i - y_i)$:
$x = 1$: $g = 2(0 - 3) = -6.0$, $w = \mathbf{0.3}$.
$x = 2$: $g = 4(0.6 - 5) = -17.6$, $w = \mathbf{1.18}$.
$x = 4$: $g = 8(4.72 - 9) = -34.24$, $w = \mathbf{2.892}$.
(c) $w^\star = 49/(1 + 4 + 16) = 49/21 = \mathbf{2.3333}$.
(d) One batch step reached 1.63; one epoch of SGD — the same single pass over the data — reached 2.89, overshooting 2.33 slightly. The batch step is the better *single* step but SGD took three of them for the same data cost. This is why n mediocre steps beat one good one.

U3.29 Verified.

Expected answer

t	g_t	v_t	w_t
1	−8.0000	−8.0000	1.2000
2	−5.6000	−12.8000	3.1200
3	−1.7600	−13.2800	**5.1120**

Plain GD reached $w_3 = 2.6280$; the optimum is 4. Momentum has **overshot to 5.11** — past the minimum by more than plain GD was short of it. The velocity still carries most of the huge first gradient, so step 3 is almost as large as step 1 even though the true gradient has collapsed to −1.76. It will now oscillate inward.

The alternative convention $v \leftarrow \beta v - \eta g$, $w \leftarrow w + v$ reaches the same w_t; either is fine, so long as you stay consistent within a question.

U3.30 Verified. (a) $\eta_{\text{eff}} = 0.01/0.1 = 0.1$ at $\beta = 0.9$; at $\beta = 0.99$ the same η would give $0.01/0.01 = 1.0$, ten times larger. (b) $\eta_{\text{new}} = \eta_{\text{old}}(1-\beta_{\text{new}})/(1-\beta_{\text{old}}) = 0.01 \times 0.01/0.1 = \mathbf{0.001}$. (c) Without rescaling, the effective step jumps tenfold and will very likely cross the stability ceiling $2(1+\beta)/L$ — the loss rises or goes `nan`, and the cause looks like a momentum problem when it is a learning-rate problem.

U3.31 Verified.

Expected answer

t	g_t	s_t	$\sqrt{s_t}$	step	w_t
1	−8.0000	64.0000	8.0000	−0.5000	0.5000
2	−7.0000	113.0000	10.6301	−0.3293	0.8293
3	−6.3415	153.2146	12.3780	−0.2562	**1.0854**

Step shrank from 0.5000 to 0.2562: a fall of **48.8%**. The gradient shrank from 8.0000 to 6.3415: a fall of only **20.7%**. The extra shrinkage is the denominator: s_t accumulates and never decays, so $\sqrt{s_t}$ rose from 8.0 to 12.4 and the effective learning rate fell with it. Note the first step is exactly η, because $s_1 = g_1^2$ makes the ratio $g_1/|g_1| = \pm 1$.

U3.32 Verified.

Expected answer

(a)

t	g_t	s_t	$\sqrt{s_t}$	step	w_t
1	−8.0000	6.4000	2.5298	−1.5811	1.5811
2	−4.8377	8.1004	2.8461	−0.8499	2.4310
3	−3.1380	**8.2750**	2.8766	−0.5454	**2.9764**

(b) $\eta/\sqrt{1-\gamma} = 0.5/\sqrt{0.1} = 0.5/0.3162 = \mathbf{1.5811}$ — matches the first step exactly, since $s_1 = (1-\gamma)g_1^2$ gives $\sqrt{s_1} = |g_1|\sqrt{1-\gamma}$.
(c) s_t has not yet fallen by $t = 3$ (6.40 → 8.10 → 8.28, still rising but decelerating); it falls as soon as $(1-\gamma)g_t^2 < (1-\gamma)s_{t-1}$, i.e. once $g_t^2 < s_{t-1}$ — here at $t = 4$. Adagrad's s_t can *never* fall because it is a sum of non-negative terms with no decay factor. That single difference is the whole of RMSprop.

U3.33 Verified.

Expected answer

t	g_t	m_t	v_t	$\hat{m}_t$	$\hat{v}_t$	$\sqrt{\hat{v}_t}$	step	w_t
1	−8.0000	−0.8000	0.064000	−8.0000	64.0000	8.0000	−0.2000	0.2000
2	−7.6000	−1.4800	0.121696	−7.7895	60.8784	7.8025	−0.1997	0.3997
3	−7.2007	−2.0521	0.173424	−7.5722	57.8658	7.6070	−0.1991	**0.5988**

Steps: 0.2000, 0.1997, 0.1991 — essentially constant at $\eta = 0.2$ regardless of the gradient's size. Adam sets the *relative* step between coordinates; the overall scale remains η's job. Note $\hat{m}_1 = g_1$ and $\hat{v}_1 = g_1^2$ exactly.

U3.34 Verified.

Expected answer

(a) Uncorrected steps: $\mathbf{-0.6325}$, $\mathbf{-0.8430}$, $\mathbf{-0.9586}$, giving $w = 0.6325,\ 1.4755,\ 2.4341$.
(b) $(1-\beta_1)/\sqrt{1-\beta_2} = 0.1/\sqrt{0.001} = 0.1/0.031623 = \mathbf{3.1623}$. The corrected first step was 0.2000; the uncorrected was $0.6325 = 0.2 \times 3.1623$. Exactly the predicted factor.
(c) With $m_0 = v_0 = 0$ and a constant g, $m_t = (1-\beta_1^t)g$ and $v_t = (1-\beta_2^t)g^2$. The step uses $m/\sqrt{v}$, so the two biases do not cancel: they enter as $(1-\beta_1^t)$ over $\sqrt{1-\beta_2^t}$. If both moments shrank by the *same* factor the ratio would be untouched and no correction would be needed. It is the mismatch — $\beta_1 = 0.9$ against $\beta_2 = 0.999$ — that makes the early uncorrected steps 3.16× too large.

U3.35 Verified. (a) Require $1/(1-\beta^t) \le 1.01$, i.e. $\beta^t \le 1/101$, i.e. $t \ge \log(1/101)/\log\beta$. (b) $\beta = 0.9 \to t \ge 43.8$, so $\mathbf{t = 44}$; $\beta = 0.99 \to \mathbf{t = 460}$; $\beta = 0.999 \to t \ge 4612.8$, so $\mathbf{t = 4613}$. (c) A 2000-step run *never* escapes the second moment's bias: correction is active for the entire training run, not just a warm-up. Switching it off would corrupt every step.

U3.36 Verified. $s_1 = 0.1 \times 64 = 6.4$; $\sqrt{s_1+\varepsilon} = 2.5298$. Then $\Delta w_1 = \sqrt{\varepsilon} \times 8/2.5298$:

ε	$\sqrt{\varepsilon}$	Δw_1
10^{-6}	10^{-3}	$\mathbf{3.1623 \times 10^{-3}}$
10^{-8}	10^{-4}	$\mathbf{3.1623 \times 10^{-4}}$
10^{-12}	10^{-6}	$\mathbf{3.16 \times 10^{-6}}$

$\Delta w_1 \propto \sqrt{\varepsilon}$ exactly. So Adadelta does have a hyperparameter controlling its step scale — it is simply spelled ε and presented as a numerical-stability constant. The claim "no learning rate" is true of the *form* of the update and misleading about the practice.

U3.37 Verified.

B	$1/\sqrt{B}$	work per update
1	1.0000	1×
4	0.5000	4×
16	0.2500	16×
64	0.1250	64×
256	0.0625	256×

Doubling B reduces noise by a factor $\sqrt{2} = 1.414$ — that is, by **29%** — for **100%** more work. Conclusion: there is a broad flat optimum. Start at $B = 32$, use powers of two, and let memory rather than statistics decide the upper limit; tune η before B. *2.5 table, 1 for the $\sqrt{2}$/29% figure, 1.5 for the conclusion.*

U3.38 Batch: $B = n$, 1 update/epoch, no gradient noise, $O(np)$ per update, vectorises well. Mini-batch: $B = 32$–512, n/B updates, moderate noise, $O(Bp)$, vectorises well. Stochastic: $B = 1$, n updates, large noise, $O(p)$, does *not* vectorise. Comments should note that mini-batch is strictly better than both extremes, which is why it is the universal default.

U3.39 SGD: $w \leftarrow w - \eta g$. Momentum: adds a velocity — $v \leftarrow \beta v + g$, $w \leftarrow w - \eta v$ — adapting the *direction*. Adagrad: adds per-coordinate scaling by $\sqrt{\sum g^2}$ — adapting the *size*. RMSprop: replaces Adagrad's sum with an EWMA so the denominator can shrink. Adam: puts momentum's first moment on top of RMSprop's second, plus bias correction.

U3.40 Adagrad: $s_i = \sum_t g_{i,t}^2$ (a sum). RMSprop: $s_i = \gamma s_i + (1-\gamma) g_i^2$ (an EWMA). Adam: $s_i = \hat{v}_i$, an EWMA with bias correction, and additionally the numerator g_i is replaced by $\hat{m}_i$. So Adam is the only one that also changes the numerator — worth saying, and worth a mark.

U3.41 Plain SGD's step $-\eta g$ has units $[\eta][J]/[w]$ — it depends on how the loss is scaled. Adagrad's and RMSprop's $-\eta g/\sqrt{s}$ has units $[\eta]$, because $\sqrt{s}$ cancels g exactly. Newton's $-H^{-1}g$ has units $[w]$ — dimensionally correct, and therefore the only one whose step is genuinely a distance in parameter space. It is not used because forming and inverting H costs $O(p^3)$.

U3.42 A dense feature is updated every batch and its accumulated squared gradient grows fast, so it wants a small step; a feature firing in 18 rows of 4000 is updated rarely and needs a large step when it is. One scalar η must serve both and will be wrong for one of them. Reach for **Adagrad**: because s_i accumulates *per coordinate*, the rare feature's denominator stays small and its effective step stays large — measured at 31.8× the median dense step.

U3.43 Standardising changes the Hessian's eigenvalues and so its condition number — measured at $470 \rightarrow 1.17 \times 10^8$ when one feature was multiplied by 1000, with the stability limit $2/L$ collapsing from 0.2485 to 4.42×10^{-4}. Since κ sets the convergence rate and $2/L$ sets the largest usable η, scaling is a decision about the optimisation problem, not about tidiness.

U3.44 (1) Standardise. (2) $B = 32$. (3) Find η by bisection on a log scale — try $10^{-1}, 10^{-2}, 10^{-3}$, take the largest that does not diverge, back off by 3×. (4) Add momentum $\beta = 0.9$, dividing η by 10 if it was tuned without. (5) Watch the loss curve and read its signature. (6) Decay η at the end.

§C — Ten-mark questions

U3.45 Coverage: the update rule; the four regimes with the multiplier $|1 - \eta f''|$ evaluated in each; the derivation of $2/L$ (the iterate contracts iff the multiplier has magnitude below 1); the four loss-curve signatures; bisection on a log scale. Worked example: U3.26 or U3.27 numbers. *The derivation of $2/L$ is the one part students omit; require it.*

U3.46 Expected chain, each with problem/fix/rule/cost: batch (exact but $O(np)$ per update) → SGD (cheap steps, but a noise floor) → mini-batch (noise as $1/\sqrt{B}$, vectorises; introduces B) → momentum (ravines; introduces β and overshoot) → Adagrad (per-coordinate scale; stalls) → RMSprop (EWMA fixes the stall; loses the convergence proof) → Adam (adds the first moment and bias correction; three more hyperparameters).

U3.47 Coverage: the ravine and κ; $v \leftarrow \beta v + g$; the unrolled EWA $v_t = \sum_j \beta^j g_{t-j}$ with total weight $1/(1-\beta)$; $\eta_{\text{eff}} = \eta/(1-\beta)$; the ceiling $\eta < 2(1+\beta)/L$; overshoot with

numbers (U3.29 gives $w_3 = 5.11$ against an optimum of 4); rate $1 - 2/\kappa \to 1 - 2/\sqrt{\kappa}$; and the caution that momentum does not help when $\kappa \approx 1$.

U3.48 Coverage: the two moments and their roles (direction / scale); the constant-gradient derivation $m_t = (1 - \beta_1^t)g$; the first-step result $\pm\eta$; the 3.1623× error if correction is omitted, growing to 6.57× at $t = 12$; the defaults $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$, $\eta = 10^{-3}$; and the misconception that "adaptive" means schedule-free.

U3.49 A four-way table plus at least two hand-computed steps of two of them. Expect: Adagrad accumulates a sum, never decays, first step η, one hyperparameter, fixes per-coordinate scale, right for sparse data. RMSprop accumulates an EWMA, decays, first step $\eta/\sqrt{1-\gamma}$, adds γ, fixes the stall, right for non-stationary objectives. Adadelta accumulates two EWMAs, first step $\propto \sqrt{\varepsilon}$, fixes the units, right when you cannot tune η. Adam accumulates both moments with correction, first step η, adds β_1, β_2, is the general default.

U3.50 Coverage: the claim that final performance differs by about one per cent on a tuned convex problem; the sensitivity experiment — sweep η over four decades, count the grid points reaching within 5% of the optimum, report the width in decades (Adagrad 2.24 against SGD 1.31); and why a wider window matters more in practice than a marginally better optimum, because tuning budget is the scarce resource.

§D — Statements

U3.51 **False.** On the diabetes benchmark SGD with momentum finished *best* (0.4846) and Adam *last* (0.4929) over 30 epochs. With a tuned η the choice of optimiser is worth about one per cent; Adam's real advantage is robustness to η, not final loss.

U3.52 **False.** Adam adapts the *relative* step between coordinates; the overall scale is still η. Measured: with decay off, the loss got *worse* between epoch 5 (0.4888) and epoch 30 (0.4929); with decay on it improved to 0.4850.

Unit IV — Data preprocessing

§A — Two-mark questions

U4.1 Data cleaning, data integration, data reduction, data transformation.

U4.2 $P = (1-p)^d$. At $p = 0.05$, $d = 30$: $0.95^{30} = \mathbf{0.2146}$ — 79% of rows lost.

U4.3 Numeric: mean (symmetric, few holes), median (skewed or outlier-prone), a constant with an indicator (missingness may be informative), KNN or MICE (holes are predictable from other columns). Categorical: most-frequent, or an explicit `"Missing"` level.

U4.4 $\sigma_{\text{after}}/\sigma_{\text{before}} = \sqrt{m/n} = \sqrt{1-p}$, where m of n are observed.

U4.5 MCAR: $P(R)$ depends on nothing (a dropped test tube). MAR: depends on observed columns only (BMI recorded only for high-BP patients). MNAR: depends on the missing value itself (the heaviest patients decline to be weighed).

U4.6 MAR — a multivariate imputer recovered 88% of the bias (+1.3473 → −0.1588). MNAR — nothing in the data can help, because the information needed is precisely

what is absent.

U4.7 A binary column recording *that* a value was missing. It is leakage when the reason for the hole will not exist at deployment — ask when in the real timeline the missingness is decided.

U4.8 $|z_i| = |x_i - \bar{x}|/s > 3$; $x_i < Q_1 - 1.5\,\mathrm{IQR}$ or $x_i > Q_3 + 1.5\,\mathrm{IQR}$; $|M_i| = |0.6745(x_i - \tilde{x})/\mathrm{MAD}| > 3.5$.

U4.9 z-score **0%** (one point moves both $\bar{x}$ and s); Tukey/IQR **25%**; modified z **50%** (both statistics are medians).

U4.10 An extreme value inflates the standard deviation it is measured against, so its own z-score shrinks and it hides behind its own effect. With k contaminants present, each masks the others.

U4.11 $|z_1| \le (n-1)/\sqrt{n}$. It exceeds 3 first at $\mathbf{n = 11}$ (3.0151); below that the $|z| > 3$ test is not merely weak, it is *inert* — `False` for every possible dataset.

U4.12 0.6745 is the 0.75 quantile of the standard Normal, so MAD $\approx 0.6745\sigma$ for Normal data and the modified z is on the same scale as an ordinary z. The multiplier 1.5 makes $Q_3 + 1.5\,\mathrm{IQR} \approx 2.70\sigma$, flagging about 0.70% of Normal data.

U4.13 Skew is a property of the true distribution; contamination is a property of the recording. An outlier rule applied to a skewed variable is mostly a skewness detector — on `'area error'` the IQR fence flagged 11.4% of rows, falling to 1 row after Yeo–Johnson. **Transform first, then detect.**

U4.14 Leave alone (it is real and it matters); trim/drop (it is provably an error); winsorise or cap at a fence (keep the row, bound the influence); use a robust method instead (`RobustScaler`, MAE, Huber).

U4.15 Equal-width cuts the *range* into intervals of equal size; equal-depth puts an equal *count* in each bin. Smoothing: by bin means, by bin medians, by bin boundaries.

U4.16 $x^{(\lambda)} = (x^\lambda - 1)/\lambda$ for $\lambda \neq 0$, $\log x$ for $\lambda = 0$. It requires $x > 0$; **Yeo–Johnson** removes that restriction.

U4.17 To make the family continuous at $\lambda = 0$: as $\lambda \to 0$, $(x^\lambda - 1)/\lambda \to \log x$ by l'Hôpital. Without the -1 and the division, the limit would be the constant 1.

U4.18 By *maximum likelihood* — `PowerTransformer` and `scipy.stats.boxcox` choose the λ maximising the Normal likelihood. It is **not** chosen to zero the skewness: on the same column those two criteria gave -0.4357 and -0.4688.

U4.19 `StandardScaler` $(x - \bar{x})/\sigma$, mean 0 sd 1, unbounded. `MinMaxScaler` $(x - x_{\min})/(x_{\max} - x_{\min})$, exactly $[0, 1]$. `RobustScaler` $(x - \tilde{x})/\mathrm{IQR}$, median 0 IQR 1, unbounded. `MaxAbsScaler` $x/\max|x|$, $[-1, 1]$, preserves zeros.

U4.20 `MinMaxScaler` — it is *defined* by the two extremes, so one bad value sets the entire range. `RobustScaler` is safest: median and IQR are order statistics with a 25% breakdown point.

U4.21 A split asks "is $x_j \le t$?". Under $x \mapsto (x-a)/b$ with $b > 0$ the threshold moves to $(t-a)/b$ and exactly the same rows go left. Predictions, structure and accuracy are identical; only the printed thresholds become unreadable.

U4.22 Scaling changes location and width only — skewness and kurtosis are unchanged. A transformation (log, Box–Cox, Yeo–Johnson) changes the *shape*, and so changes skewness.

U4.23 Nominal categories have no order (colour, city); ordinal do (small/medium/large). Nominal → one-hot. Ordinal → ordinal encoding *with the order supplied explicitly.*

U4.24 With an intercept and all k dummies, the dummies sum to 1 in every row and duplicate the intercept, so the design matrix is rank deficient. It breaks coefficients, standard errors, t-statistics and p-values. It does *not* break predictions — both coefficient vectors gave identical predictions and $R^2 = 0.986231$.

U4.25 Alphabetical. So `large` sorts before `medium` before `small`, producing the codes 0, 1, 2 in exactly the wrong order — and it does so silently, with no error and no warning.

U4.26 It emits an all-zero row for a category not seen during `fit`. Without it, `OneHotEncoder` raises `ValueError: Found unknown categories` at `transform` time — in production, or inside a CV fold.

U4.27 Dimensionality reduction (fewer columns), numerosity reduction (fewer rows), and compression.

U4.28 As d grows, the ratio of the farthest to the nearest distance tends to 1 — measured 9135 at $d = 1$ against 1.11 at $d = 1000$. Every distance-based method degrades, because "nearest" stops meaning anything.

U4.29 Centre the data; form the covariance $S = \frac{1}{n-1} X_c^\top X_c$; take its eigenvalues and eigenvectors, sorted by eigenvalue; project, $T = X_c U_k$.

U4.30 λ_j *is* the variance along component j. The trace equals $\sum_j \lambda_j$ — the total variance, which a rotation moves around but never creates or destroys.

U4.31 PCA maximises variance, and variance has units. Unstandardised on breast_cancer, PC1 explained 98.20% and was essentially the single column `worst area` (sd 568.9); standardised, PC1 explained 44.27% and drew on many columns. Without standardising you are ranking columns by their units.

U4.32 PCA keeps *combinations* of all the features, not a subset of them. On breast_cancer, 26 of 30 loadings on PC1 exceeded 0.10. Saying you "only need ten components" does not mean you only need ten measurements — you still need all thirty to compute them.

U4.33 Variance is not relevance. In the designed example a $\mathcal{N}(0, 10^2)$ noise column carried 99.05% of the variance while the label depended entirely on a $\mathcal{N}(0, 1)$ column: logistic regression scored 0.9867 on both features and **0.5217** on PCA's one component. Supervised alternative: **LDA** (or PLS).

U4.34 *Filter* (variance threshold, ANOVA F, mutual information) — cheapest, but blind to interactions and redundancy. *Wrapper* (RFE, forward selection) — most expensive,

and can overfit the selection. *Embedded* (L1/lasso) — nearly free, but the answer is specific to that model.

U4.35 F measures *linear* association; mutual information measures *any* statistical dependence. A feature entering only through $|b|$ ranked 1st by MI and 8th of 10 by F ($F = 0.122$, below several pure-noise columns).

U4.36 L1 has a corner at zero, so the optimum frequently lands exactly on it and coefficients become *exactly* zero — selection. L2 is smooth, so coefficients shrink towards zero without reaching it — ridge at $\alpha = 1$ kept all 10 non-zero while lasso at $\alpha = 5$ kept 5.

U4.37 Any step that *learns* anything from the data — a mean, a median, a category vocabulary, a feature subset, a hyperparameter, a resampled row — must be fitted on the training portion only, and therefore belongs inside the cross-validation loop.

U4.38 **Feature selection**, by a wide margin, because it looks at y. Measured: scaling on everything leaked +0.0034 at $n_{\text{train}} = 20$ and +0.0002 at 427; selection on everything leaked +0.3133. Scaling and PCA never see the labels; selection does.

U4.39 So that the class proportions of the whole are preserved in each part. Without it a fold can contain no minority rows at all, making recall undefined. Argument: `stratify=y`, or use `StratifiedKFold`.

U4.40 `shuffle=False`. On a file sorted by label, the folds partition by class: measured, 200 rows with 10 positives gave positives per test fold $[10, 0, 0, 0, 0]$, mean recall 0.0000 and mean accuracy 1.0000 with no error message.

U4.41 The unit that could not have produced another row — the level at which observations are genuinely independent. Not the row when: many windows per patient; many transactions per cardholder; many sentences per document.

U4.42 `GroupKFold`; `TimeSeriesSplit`; `StratifiedKFold`; `StratifiedGroupKFold`.

U4.43 k fits; each on $n(1 - 1/k)$ rows; total row-fits $n(k - 1)$.

U4.44 Cost: n fits — on breast_cancer, 2.07 s against 0.02 s for 5-fold, a factor of 98 for the same accuracy (0.9789 both). Variance: the across-fold sd is $\sqrt{p(1-p)} = 0.1437$, which is a property of scoring one row at a time, not of the estimator, and tells you nothing.

U4.45 It is the spread of the k fold scores, not the standard error of their mean. LOOCV's 0.1437 comes from each fold being a single 0/1 outcome. Report the mean with the spread across repeated CV runs if you need an uncertainty.

U4.46 The validation set *selects* — look at it as often as you like. The test set *estimates* — look at it once, after everything is fixed.

U4.47 It is the maximum over the grid of noisy scores, so it is a selection and is biased upward — the expected maximum of m noisy scores grows like $\mu + \sigma\sqrt{2 \ln m}$. Measured on pure noise: 1 combination gave optimism exactly 0.0000, 60 combinations gave +0.0783. The estimate is **nested cross-validation**, or a held-out test set touched once.

U4.48 acc $= \frac{TP+TN}{TP+TN+FP+FN}$; prec $= \frac{TP}{TP+FP}$; rec $= \frac{TP}{TP+FN}$; $F_1 = \frac{2TP}{2TP+FP+FN}$; bal $= \frac{1}{2}\left(\frac{TN}{TN+FP} + \frac{TP}{TP+FN}\right)$.

U4.49 ROC–AUC: **0.5** always. PR–AUC: the **positive rate** — 0.01 on a 1%-positive problem. This is why a PR–AUC of 0.19 there can be a real result.

U4.50 $x_{\text{new}} = A + u(B - A)$, $u \sim \mathcal{U}(0, 1)$, where A is a minority point and B is one of A's k nearest *minority* neighbours.

U4.51 Any two: categorical features (the interpolation is meaningless); multi-modal minority classes (the line between two modes passes through empty space); minority points sitting inside a majority region (synthetic points land squarely in majority territory).

U4.52 Inside each fold, on the *training portion only*, after the transformers are fitted and immediately before the model fit. Earlier, and copies of the same row land in both training and validation — measured F_1 0.8942 against 0.1133, an overstatement of +0.7809, with 100% of the minority test rows being exact copies of training rows.

U4.53 **Moving the decision threshold** using the precision–recall curve. It is cheaper (no refitting, no extra rows), deterministic, and leaves the model's probabilities calibrated.

§B — Five-mark questions

U4.54 **Verified.**

Expected answer

(a) Observed $48, 55, 60, 67, 72, 78$: mean $= 380/6 =$ **63.3333**; median $= (60 + 67)/2 =$ **63.5**; sd ddof=1 = **11.1295**, ddof=0 = **10.1598**.
(b) Mean-imputed (two extra values at 63.3333): mean = **63.3333** (unchanged, by construction), population sd = **8.7987**.
(c) Median-imputed (two extra at 63.5): mean = **63.375**, population sd = **8.7990**.
(d) $m = 6$, $n = 8$, so $\sqrt{m/n} = \sqrt{0.75} =$ **0.8660**. Ratio of imputed to observed population sd $= 8.7987/10.1598 =$ **0.8660**. *Why:* the imputed values sit exactly at the mean, so they contribute *zero* to $\sum(x_i - \bar{x})^2$ while still counting in the denominator n. The sum of squares is unchanged and the divisor grew from m to n, giving a variance factor of exactly m/n and an sd factor of $\sqrt{m/n}$.

U4.55 **Verified.**

Expected answer

(a)

p	$d = 5$	$d = 20$	$d = 30$	$d = 50$
0.01	0.9510	0.8179	0.7397	0.6050
0.05	0.7738	0.3585	**0.2146**	0.0769
0.10	0.5905	0.1216	0.0424	**0.0052**

(b) $(0.95)^d = 0.5 \Rightarrow d = \log 0.5 / \log 0.95 =$ **13.5** columns.

(c) $0.9^{50} = 0.005154$, so $1000/0.005154 =$ **194,033** raw rows.
(d) Listwise deletion is right when the rows are few relative to n, the missingness is MCAR (so the survivors are unbiased), and you want an analysis whose validity does not depend on an imputation model. It is never free: with $d = 30$ and $p = 5\%$ you are discarding 79% of your evidence.

U4.56 Verified.

Expected answer

(a) $n = 12$. Mean $= 452/12 =$ **37.6667**; sd (ddof=1) = **39.6584**; median = **21.0**; absolute deviations from the median are $1, 0, 2, 1, 1, 0, 2, 2, 1, 1, 99, 104$, so MAD = **1.0**.
(b) $Q_1 =$ **20.0**, $Q_3 =$ **22.25**, IQR = **2.25**; fences $20 - 3.375 =$ **16.625** and $22.25 + 3.375 =$ **25.625**.
(c) For $x = 120$: $z = (120 - 37.6667)/39.6584 =$ **2.0761** — under $|z| > 3$, **not flagged**. Tukey: $120 > 25.625$ — **flagged**. Modified $z = 0.6745(120-21)/1.0 =$ **66.78** — **flagged**, by a factor of 19 over the threshold. Flag counts across the column: $|z| > 3$ **0**; IQR fence **2**; $|M| > 3.5$ **2**.
(d) Standard deviation of the ten genuine readings alone: **1.3375**. The value 120 would then score $z = (120 - 20.7)/1.3375 =$ **74.24**. The discrepancy is **masking**: the two contaminants inflated s from 1.34 to 39.66 — a factor of 30 — so each hides behind the variance the *pair* of them created. Note also the ceiling: with $n = 12$ the largest attainable $|z|$ is $11/\sqrt{12} = 3.1754$, so the rule had almost no room to fire even in principle.

U4.57 Verified.

Expected answer

n	$(n-1)/\sqrt{n}$	can $\lvert z\rvert > 3$ fire?
5	1.7889	**no**
8	2.4749	**no**
10	2.8460	**no**
11	3.0151	yes
20	4.2485	yes

Derivation. Let $d_i = x_i - \bar{x}$. Then $\sum d_i = 0$ and $\sum d_i^2 = (n-1)s^2$. From $\sum_{i \geq 2} d_i = -d_1$ and Cauchy–Schwarz, $\sum_{i \geq 2} d_i^2 \geq d_1^2/(n-1)$. Hence $(n-1)s^2 \geq d_1^2 + d_1^2/(n-1) = d_1^2 \frac{n}{n-1}$, giving $|z_1| = |d_1|/s \leq (n-1)/\sqrt{n}$.
Consequence. On batches of eight, `if abs(z) > 3` evaluates to `False` for *every possible input*. The line is not a weak test; it is dead code that looks like a test, and it will never raise an alert or an error.

1.5 table, 2 derivation, 1.5 consequence.

U4.58 Verified.

Expected answer

(a)

bin	values	mean	median	boundaries
1	$5, 8, 14$	9	8	5, 14
2	$17, 20, 26$	21	20	17, 26
3	$27, 30, 36$	31	30	27, 36

(b) By means: $9, 9, 9,\ 21, 21, 21,\ 31, 31, 31$. By medians: $8, 8, 8,\ 20, 20, 20,\ 30, 30, 30$. By boundaries (each value to its nearer boundary): $5, 5, 14,\ 17, 17, 26,\ 27, 27, 36$.
(c) Equal-width on $[5, 36]$: three intervals of width $31/3 = \mathbf{10.33}$ — $[5, 15.33)$, $[15.33, 25.67)$, $[25.67, 36]$ — with bin sizes $\mathbf{3}, \mathbf{2}, \mathbf{4}$ rather than $3, 3, 3$.
(d) Gains: local noise is smoothed away, and a linear model can express a non-monotone relationship it otherwise could not (measured $+0.0914$ R^2). Costs: within-bin resolution is destroyed, the cut points are arbitrary, and a tree — which can already find its own thresholds — is actively harmed (-0.0534).

U4.59 Verified.

Expected answer

treatment	mean	sd	comment
leave alone	37.6667	39.6584	useless
winsorise 10/90	35.6500	34.8891	**did almost nothing**
cap at IQR fence	21.5208	2.2669	close
drop by modified z	**20.7000**	**1.3375**	exact

Truth: mean 20.7, sd 1.34.
Ranking: drop-by-modified-z (recovers the truth exactly, because the rule identified precisely the two contaminants); cap at the fence (close, but the two capped values sit at 25.625 and still pull the mean up by 0.8); winsorise 10/90 (a 10% tail on $n = 12$ is 1.2 values, so it clips one contaminant down towards the other and leaves the damage almost intact); leave alone.
The lesson: a fixed percentile is a *guess* about how much dirt there is. When the guess is wrong the treatment does nothing while appearing to have been applied.

U4.60 Verified. (a) Gaps 4, 12, 36 — growing by a factor of 3. (b) log: 0.6931, 1.7918, 2.8904, 3.9890; gaps **1.0986**, **1.0986**, **1.0986**. (c) Exactly $\ln 3 = 1.0986$, because $\log(3x) = \log x + \log 3$: a constant *ratio* becomes a constant *difference*. (d) Most quantities with a long right tail are multiplicative in origin — incomes, populations, areas, prices. The log converts that multiplicative structure into the additive structure a linear model assumes, and it does so with a single rung of the ladder of powers and no fitted parameter.

U4.61 Verified.

Expected answer

λ	$x = 1$	4	16	64
0.5	0.0000	2.0000	6.0000	14.0000
-0.5	0.0000	1.0000	1.5000	1.7500
0.001	0.0000	1.3873	2.7764	4.1675
$\log x$	0.0000	1.3863	2.7726	4.1589

The $\lambda = 0.001$ row agrees with $\log x$ to three decimals and would agree to more as $\lambda \to 0$. This is why the $\lambda = 0$ case of Box–Cox is *defined* to be $\log x$: the -1 and the division by λ exist precisely so that the limit is the logarithm rather than the constant 1.
Note also $\lambda = -0.5$ compresses the range hardest (1 to 1.75) — the further down the ladder, the more a long right tail is pulled in.

U4.62 Verified.

Expected answer

(a) Mean = 48/8 = **6.0**; population sd = **2.0**; min **3**, max **10**; median = **5.5**; Q_1 = **5.0**, Q_3 = **6.5**, IQR = **1.5**.
(b)

	3	5	5	5	6	6	8	10
Standard	-1.5	-0.5	-0.5	-0.5	0	0	1.0	2.0
MinMax	0	0.2857	0.2857	0.2857	0.4286	0.4286	0.7143	1.0
Robust	-1.6667	-0.3333	-0.3333	-0.3333	0.3333	0.3333	1.6667	3.0
MaxAbs	0.3	0.5	0.5	0.5	0.6	0.6	0.8	1.0

(c) Standard: mean 0, sd 1, *unbounded*. MinMax: exactly $[0, 1]$. Robust: median 0, IQR 1, unbounded. MaxAbs: $[-1, 1]$, and zeros stay zero.

U4.63 Verified.

Expected answer

(a) With 400 appended ($n = 9$): mean 6.0 → **49.7778**; population sd 2.0 → **123.8366**; median 5.5 → **6.0**; IQR 1.5 → **3.0**.
(b) Span occupied by the eight good values:

scaler	before	after	compression
Standard	3.5000	0.0565	**61.9**×
MinMax	1.0000	0.0176	**56.7**×
Robust	4.6667	2.3333	**2.0**×

(c) **RobustScaler** survives. Mean and sd have 0% breakdown — one point moves both without limit — and min/max are the extremes themselves, so one bad value *defines* MinMax's range. Median and IQR are order statistics: the typo moved the median by 0.5 and the IQR by 1.5, so the good values keep most of their spread.
(d) Its output is *guaranteed* to lie in $[0, 1]$, so it passes every sanity check you might run on it. Nothing in the output announces that eight of nine rows are now crammed into a window of width 0.018 — the numbers look perfectly scaled, and the information has already been destroyed.

U4.64 Verified.

Expected answer

(a) $d(A,B)^2 = 25^2 + 5000^2 = 625 + 25{,}000{,}000 =$ **25,000,625** — income supplies **99.998%**. $d(A,C)^2 = 1^2 + 400{,}000^2 = 1 + 1.6 \times 10^{11} =$ **160,000,000,001** — income supplies **100.000%**.
(b) Column means (38.6667, 635,000), population sds (11.5566, 187,394.4). Standardised: $A = (-0.7499, -0.7204)$, $B = (1.4133, -0.6937)$, $C = (-0.6634, 1.4141)$. $d(A,B)^2 = 4.6797 + 0.0007 =$ **4.6804**; $d(A,C)^2 = 0.0075 + 4.5562 =$ **4.5637**.
(c) Raw: A's nearest neighbour is **B** (2.5×10^7 against 1.6×10^{11}). Standardised: **C** ($4.5637 < 4.6804$). **The neighbour changed.**
(d) Choosing not to scale is choosing to weight every feature by its own standard deviation. Here that means declaring one rupee of income as important as one year of age — a claim nobody would defend if stated aloud, but which is made silently by every unscaled distance computation.

U4.65 Verified.

Expected answer

(a) $\bar{x} = 1$; $\bar{y} = 115/3 =$ **38.3333**. $S_{xy} = (-1)(30 - 38.3333) + 0 + (1)(15 - 38.3333) = 8.3333 - 23.3333 =$ **−15**. $S_{xx} = 1 + 0 + 1 =$ **2**.
(b) Slope $= -15/2 =$ **−7.5**; intercept $= 38.3333 + 7.5 =$ **45.8333**.
(c) Predictions: blue **45.8333**, green **38.3333**, red **30.8333** — against true means 30, 70, 15. SSE $= 15.8333^2 + 31.6667^2 + 15.8333^2 =$ **1504.1667**; SST $= 8.3333^2 + 31.6667^2 + 23.3333^2 =$ **1616.6667**; $R^2 = 1 - 1504.1667/1616.6667 =$ **0.0696**.
(d) One-hot achieves $R^2 =$ **1.0** (three dummies can reproduce three group means exactly). The ordinal code asserted that green sits exactly halfway between blue and red on the target scale, and that the step blue→green equals green→red. Both are false — green is the *highest* group. A linear model can only add its inputs up, so it believes the assertion and fits a straight line through means that are not collinear, recovering 7% of the variance instead of all of it. A decision tree on the same ordinal column would score 1.0, because it can split twice and never uses the spacing.

U4.66 Verified. (a) **5** columns; **4** with `drop='first'`. (b) Intercept plus 5 dummies is a $n \times 6$ matrix of rank **5** — the dummies sum to the all-ones column in every row, so one column is a linear combination of the others. (c) It breaks coefficients, standard errors, t-statistics and p-values — they become non-unique. It does **not** break predictions or R^2: two entirely different coefficient vectors gave a maximum prediction difference of 0.00 and identical $R^2 = 0.986231$. (d) Drop when anyone will *interpret* the coefficients, or when the model is unregularised OLS. Do not drop for regularised models (the penalty is no longer symmetric across levels) or for trees (which do not have an intercept to collide with).

U4.67 Verified. $\binom{d+2}{2} - 1$ and $\binom{d+3}{3} - 1$:

d	degree 2	degree 3
2	**5**	9
10	**65**	285
30	**495**	5455
100	**5150**	176850

The count grows like d^k. At $d = 30$, degree 2 already produces 495 columns — and on the diabetes data, expanding 10 raw features to 65 degree-2 features made ridge *worse*, R^2 falling from 0.4822 to 0.4179. Generating all interactions is not feature engineering; it is buying variance in bulk and hoping regularisation pays for it. Engineer a feature because you have a reason.

U4.68 Verified.

Expected answer

(a) Mean $= (3, 3)$. Centred: $(-2, -2)$, $(-1, 0)$, $(0, -1)$, $(1, 2)$, $(2, 1)$.
(b) $\sum x_c^2 = 4 + 1 + 0 + 1 + 4 = 10$; $\sum y_c^2 = 4 + 0 + 1 + 4 + 1 = 10$; $\sum x_c y_c = 4 + 0 + 0 + 2 + 2 = 8$. With divisor $n - 1 = 4$: $S = \begin{pmatrix} 2.5 & 2.0 \\ 2.0 & 2.5 \end{pmatrix}$.
(c) The diagonals are equal, so the eigenvectors are $\frac{1}{\sqrt{2}}(1, 1)$ and $\frac{1}{\sqrt{2}}(1, -1)$ and the eigenvalues are $a \pm b = 2.5 \pm 2.0$: $\lambda_1 = \mathbf{4.5}$, $\lambda_2 = \mathbf{0.5}$. Trace check: $2.5 + 2.5 = 5.0 = 4.5 + 0.5$. ✓
(d) Variance explained: $4.5/5 = \mathbf{0.90}$ and $0.5/5 = \mathbf{0.10}$.
(e) $u_1 = \frac{1}{\sqrt{2}}(1, 1)$, so a score is $(x_c + y_c)/\sqrt{2}$: $\mathbf{-2.8284}$, $\mathbf{-0.7071}$, $\mathbf{-0.7071}$, $\mathbf{2.1213}$, $\mathbf{2.1213}$. Their sample variance is $(8 + 0.5 + 0.5 + 4.5 + 4.5)/4 = 18/4 = \mathbf{4.5} = \lambda_1$. ✓
(f) Reconstruction = score $\cdot$ u_1 + mean: $(1, 1)$, $(2.5, 2.5)$, $(2.5, 2.5)$, $(4.5, 4.5)$, $(4.5, 4.5)$. Note $(2, 3)$ and $(3, 2)$ both collapse onto $(2.5, 2.5)$ — one component cannot tell them apart. Total squared error $= 0 + 0.5 + 0.5 + 0.5 + 0.5 = \mathbf{2.0}$, and $(n-1)\lambda_2 = 4 \times 0.5 = \mathbf{2.0}$. ✓

U4.69 Verified. Total = 10.0.

Expected answer

(a) Ratios: **0.48**, **0.24**, **0.13**, **0.08**, **0.04**, **0.03**.
(b) Cumulative: **0.48**, **0.72**, **0.85**, **0.93**, **0.97**, **1.00**.
(c) 80% needs $k = \mathbf{3}$; 90% needs $k = \mathbf{4}$; 95% needs $k = \mathbf{5}$.
(d) Relative reconstruction error $= 1 -$ (variance kept), so **0.15**, **0.07**, **0.03**. No computation is needed because variance kept and relative error removed *add to exactly one*: total squared reconstruction error is $(n-1)\sum_{j>k}\lambda_j$ and the total variance is $(n-1)\sum_j \lambda_j$, so their ratio is $1 - \sum_{j\le k}\lambda_j / \sum_j \lambda_j$.

U4.70 Verified. (a) All **three** positives are in the test set and **none** in training: the model cannot learn the positive class at all, and the test set is 75% positive against a 25% population. (b) $P(0) = \binom{9}{4}/\binom{12}{4} = 126/495 = \mathbf{0.2545}$ — better than one chance in four. (c) Exactly one positive and three negatives in the test set, *every* time: $3 \times \frac{1}{3} = 1$ and $9 \times \frac{1}{3} = 3$. (d) Recall is 0/0; scikit-learn returns **0.0** and emits a warning, which is easy to miss — so a fold with no positives silently drags the mean recall down while accuracy reads 1.0000.

U4.71 Verified.

Expected answer

(a) $n = 10{,}000$. accuracy $= 9910/10000 =$ **0.9910**; precision $= 10/70 =$ **0.1429**; recall $= 10/40 =$ **0.2500**; $F_1 = 20/110 =$ **0.1818**; balanced $= \frac{1}{2}(9900/9960 + 10/40) = \frac{1}{2}(0.9940 + 0.2500) =$ **0.6220**.
(b) Always-negative: $9960/10000 =$ **0.9960**.
(c) The *always-negative* model scores higher — 0.9960 against 0.9910. A model that has learnt something real is beaten on accuracy by a model that has learnt nothing, because 99.6% of the answer is "no". Accuracy on an imbalanced problem measures the base rate, not the model.
(d) **Recall** at a stated threshold, alongside **PR–AUC** (chance baseline 0.004 here, not 0.5). Justification: a false negative is a patient with the disease sent home; a false positive is a patient given a further test. The costs differ by orders of magnitude, so the headline metric must be one that penalises missed positives specifically — and recall of 0.25 says three in four cases are being missed, which accuracy of 0.991 completely conceals.

U4.72 Verified. $n = 5000$; row-fits $= n(k-1)$.

Expected answer

scheme	fits	rows/fit	row-fits	time
2-fold	2	2500	5,000	0.5 s
5-fold	5	4000	20,000	**2.0 s**
10-fold	10	4500	45,000	4.5 s
LOOCV	5000	4999	24,995,000	**2499.5 s ≈ 42 min**

At 0.4 s per 4000-row fit the unit cost is 10^{-4} s per row-fit.
Choice: 5-fold. LOOCV costs 1250× more for an estimate that is not better — measured on breast_cancer, 5-fold and LOOCV both returned 0.9789. Use $k = 5$ or 10; do not use LOOCV because it sounds thorough.

U4.73 Verified. (a) Validation fold: $5000/5 =$ **1000** rows with **80** positives. Training portion: **4000** rows with **320** positives. (b) Oversampling the 4000 training rows (3680 majority, 320 minority) to balance gives **7360** rows, $[3680, 3680]$; 3360 of the 3680 minority rows are duplicates, i.e. **91.3**%. (c) Undersampling gives **640** rows, $[320, 320]$, discarding $3360/3680 =$ **91.3**% of the majority. (d) The validation fold is **left alone** in both cases — it keeps its natural 80/1000 balance, because that is the distribution the model will meet in deployment and the only one against which a metric means anything.

U4.74 Verified. (a) $B - A = (4, 8)$, so $A + u(B - A)$ gives $u = 0 \to (\mathbf{2}, \mathbf{3})$; $0.25 \to (\mathbf{3}, \mathbf{5})$; $0.5 \to (\mathbf{4}, \mathbf{7})$; $0.75 \to (\mathbf{5}, \mathbf{9})$; $1 \to (\mathbf{6}, \mathbf{11})$. (b) The straight **line segment** joining A and B. (c) SMOTE asserts that the straight line between two minority points is itself full of minority points. (d) Any categorical or one-hot feature — a synthetic "0.4 of the way from red to blue" is not a category. Also a multi-modal minority class, where the segment between two clusters passes through majority territory.

U4.75 Verified. (a) SE $= \sqrt{0.91 \times 0.09/44} = \sqrt{0.001861} =$ **0.0431**. (b) $1/44 =$ **0.0227** — one row is worth 2.3 accuracy points, so the score can only move in steps larger than the difference between many real models. (c) SE $\propto 1/\sqrt{n}$, so halving it needs

four times the test set: **176** rows, giving SE = 0.0216. (d) With SE ≈ 0.043, the difference between two models must exceed roughly 0.12 before a single split can distinguish them — and measured over 500 seeds on wine, the same pair of models produced verdicts ranging from +0.1556 to −0.1333. You would be measuring the split, not the models.

U4.76 (a) **Resampling before the split** — a leak. (b) Random oversampling duplicates minority rows; the split then scatters those duplicates across both sides, so a row the model memorised in training reappears in the test set. The model is being asked to recognise rows it has already seen, and it does so perfectly. The measured overstatement was F_1 + 0.7638 for random oversampling and +0.7809 for SMOTE — larger than any other leak in the course. (c) Close to **100%**: when the minority is oversampled by a factor of roughly 100, almost every minority row in the data is a copy of some original, so almost every minority test row has its twin in training. The measured figure was 990 of 990, i.e. 100.0%. (d) Move the resampling inside the fold — in practice, use `imblearn.pipeline.Pipeline`, because scikit-learn's own `Pipeline` cannot hold a step that changes the number of rows.

§C — Ten-mark questions

These are the long questions. Mark for coverage, structure and at least one worked number. **An answer with no numbers anywhere caps at 6 of 10.**

U4.77 See the worked mock Section C for the full four-part breakdown — this is the same question. Key figures: 1000 validation rows / 80 positives; 4000 training rows / 320 positives; everything refitted per fold; resampling last, inside the fold; PR–AUC or recall, never accuracy at a 92% base rate.

U4.78 Coverage: $(1-p)^d$ with the 0.2146 figure; mean/median/mode and their situations; the $\sqrt{m/n}$ derivation; MCAR/MAR/MNAR with examples; MICE recovers 88% of MAR bias and nothing of MNAR (the sign flip +5.603 → −0.413 is the memorable case); the indicator and its leakage condition; and the recommendation — worry about whether you are *deleting rows*, not about which imputer, since the imputers differed by 0.003 and deletion by 5.1 at 30% missing.

U4.79 Coverage: the three rules and their breakdown points 0%/25%/50%; masking and swamping; a worked demonstration (U4.56 numbers); the $(n-1)/\sqrt{n}$ ceiling *with* the proof; skew is not contamination, so transform first; the four disposal options. Conclusion required: detection is statistics, disposal is a judgement you must defend in writing.

U4.80 Coverage: the three mechanisms (distance, gradient/conditioning, regularisation penalties); the four scalers with formulae and ranges; a worked neighbour flip (U4.64); which models change and which provably do not, with the $x \mapsto (x-a)/b$ split argument; the one-typo compression table; and the training-split rule.

U4.81 Coverage: the maximum-variance question; the four steps; λ_j as variance and the trace identity; a full by-hand example (U4.68) including $(n-1)\lambda_2$; choosing k and why there is no universal 95% rule (5 of 30 on breast_cancer, 40 of 64 on digits); standardise first, with the `worst area` failure; PCA is not selection (26 of 30 loadings above 0.10); and PCA is unsupervised (0.9867 against 0.5217).

U4.82 Coverage: a single split has SE ≈ $1/\sqrt{n_{\text{te}}}$ and was measured over 500 seeds at 0.7556

to 1.0000 on the same code; the k-fold recipe and the $n(k-1)$ cost model; k trading bias (training-set size) against time, with the learning curve flattening; LOOCV's $\sqrt{p(1-p)}$ artefact; the three failure modes and their splitters; and nested CV, with optimism growing from 0.0000 at one grid point to +0.0783 at sixty.

U4.83 Coverage: the always-negative confusion matrix; accuracy, precision, recall, F_1, balanced accuracy, ROC–AUC and PR–AUC with their chance baselines; the four remedies; SMOTE's arithmetic and its assumption; the measured result that resampling raises recall from 0.04 to about 0.78 while precision falls from 0.13 to 0.05; threshold moving as the cheaper alternative; and the inside-the-fold rule with the +0.78 F_1 overstatement.

U4.84 Expected ranking, largest error first: **resampling before the split** (F_1 + 0.78); **grouped rows split randomly** (+0.35); **feature selection on all the data** (+0.31); **time-ordered rows split randomly** (R^2 + 3.81, a ranking inversion); **row-dropping using the model then splitting** (+0.17); **scaler on all the data** (+0.0034 falling to +0.0002 as n grows); **imputer on all the data** (+0.0003, not distinguishable from zero). The correct construction in each case: put the step inside a `Pipeline` and let the cross-validator refit it per fold — except row-dropping, which cannot live in a `Pipeline` at all, and that is exactly why it is dangerous.

U4.85 Coverage: filter/wrapper/embedded with two methods each and the cost–risk of each; a filter measures the dependence it was built to measure (F vs MI, with the $|b|$ feature ranked 1st and 8th); RFE costs one fit per elimination; L1 selects and L2 shrinks; the Jaccard overlaps as low as 0.20 between selectors; and the honest finding that on breast_cancer *not one* selector beat using all thirty features.

U4.86 Coverage: what feature engineering is; combine/decompose/bin/ transform; the four-plots-of-land example where $\mathrm{corr}(\mathrm{price}, \mathrm{width}) = -0.0948$ but the product column takes R^2 from 0.6558 to 1.0000; binning helps a linear model +0.0914 and harms a tree −0.0534, so a representation is good only relative to a model; and the combinatorial blow-up as the reason automatic generation is not a substitute for a reason.

§D — Statements

U4.87 **This is the definition, not an answer.** If the question asked what the rule *does*, it is true but empty; if the question asked anything about its behaviour, it does not address the question. The substantive points are the 0% breakdown point, masking, and the $(n-1)/\sqrt{n}$ ceiling.

U4.88 **False.** PCA keeps *linear combinations* of all features, not a subset. 26 of 30 loadings on PC1 exceeded 0.10. You still need every original measurement to compute the components.

U4.89 **False.** Provably no change: a split $x_j \le t$ becomes $(t-a)/b$ under a positive affine rescaling and sends the same rows left. Measured: identical structure, identical predictions, accuracy 0.902098 both ways.

U4.90 **False.** It preserves the mean and destroys the variance: σ falls by $\sqrt{1-p}$, so every standard error is too small and every correlation is attenuated (0.5865 → 0.4536 at 40% missing). It manufactures confidence you do not have.

U4.91 **False.** Measured: 257 levels in 600 rows gave train R^2 0.9356 and test 0.6984, against 0.8081 without the id column — the encoding made the model *worse*. 82 levels appeared in training and never in test.

U4.92 **False as stated** — and this one matters, because the overclaim is common. Fitting the scaler on everything is genuinely wrong and should be fixed, but the measured effect was +0.0034 at 20 training rows and +0.0002 at 427. Report what you measured, not what you expected. The large leaks come from steps that see y.

U4.93 **False.** It costs n fits (98× 5-fold for the same 0.9789), and its across-fold sd of 0.1437 is $\sqrt{p(1-p)}$ — an artefact of scoring one row at a time, carrying no information. Use $k = 5$ or 10.

U4.94 **False.** SMOTE interpolates between existing minority points: it *asserts* that the segment between two minority examples is full of minority examples. On categorical features or a multi-modal minority class that assertion is simply wrong.

U4.95 **False.** An always-negative model scores 0.99 on the same problem and has learnt nothing. Measured, a real logistic regression scored 0.9867 — *worse* than the dummy's 0.9900 — while its PR–AUC of 0.1075 against a chance baseline of 0.0100 showed it had learnt a great deal.

U4.96 **False.** It is a coin toss with your result on it. Over 500 seeds on wine the same model scored between 0.7556 and 1.0000 — a range of 0.2444.

Unit V (part) — Regression (L8, L9, L10)

Scope, and one thing to watch

Covered: L8 least squares, L9 maximum likelihood / R^2 / adequacy / over-fitting, L10 logistic regression. **Not covered:** multiple linear regression (L11) and regularization (L12). *Note:* this material sits in L8 and L9, and L10 to **T5**. A mid-semester paper spanning L8–L10 therefore cuts across that mapping. Nothing here is wrong, but if a student asks why the bank groups L10 with L8–L9 when T5 groups it with L11–L12, the answer is that the tests follow the teaching order and the mid-semester paper follows the calendar.

§A — Two-mark questions

U5.1 $y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$. y response, x predictor, β_0 intercept, β_1 slope, ε_i error. **Greek** letters are the unknown *population* parameters and the unobservable errors; **Roman** (b_0, b_1, e_i) are the *estimates* and the observed residuals. It matters because writing "$\varepsilon_i = 0.6$" claims to have observed something unobservable.

U5.2 Linear **in the parameters**, not in x. (a) linear; (b) **not** — $\beta_0\beta_1$ is a product of parameters; (c) linear; (d) not.

U5.3 $\text{SSE}(b_0, b_1) = \sum_i (y_i - b_0 - b_1 x_i)^2$ — the sum of squared **vertical** deviations of the observed y from the fitted line. *Insist on "squared vertical"; "the distance from the line" is the perpendicular distance and is a different method.*

U5.4 $nb_0 + b_1 \sum x_i = \sum y_i$ and $b_0 \sum x_i + b_1 \sum x_i^2 = \sum x_i y_i$.

U5.5 $b_1 = S_{xy}/S_{xx}$, $b_0 = \bar{y} - b_1\bar{x}$, where $S_{xx} = \sum(x_i - \bar{x})^2$ and $S_{xy} = \sum(x_i - \bar{x})(y_i - \bar{y})$.

U5.6 Any three: the derivative of a square is linear, so the first-order conditions form a *linear* system with a closed-form solution; differentiable everywhere; the estimator is unique whenever $S_{xx} > 0$; it is the maximum likelihood estimator under Gaussian noise; and Gauss–Markov minimum variance among linear unbiased estimators. Given up: robustness — squaring hands a single bad row disproportionate influence.

U5.7 Because the model treats x as fixed and known and y as the thing with error; the residual is an error in y alone. Minimising perpendicular distance gives **total least squares** (orthogonal / major-axis regression) — which is the first principal component of the data, linking back to L6.

U5.8 $H = 2\begin{pmatrix} n & \sum x_i \\ \sum x_i & \sum x_i^2 \end{pmatrix}$, $\det H = 4nS_{xx}$. Positive definite iff $n > 0$ and $S_{xx} > 0$ — that is, iff the x_i are **not all equal**. Then SSE is strictly convex and the stationary point is the unique global minimum.

U5.9 Any dataset with all x identical, e.g. $x = (3, 3, 3, 3, 3)$. Then $S_{xx} = 0$, $\det(X^\top X) = 0$, and $b_1 = S_{xy}/S_{xx}$ is undefined — infinitely many lines fit equally well.

U5.10 $X^\top X\, b = X^\top y$, so $b = (X^\top X)^{-1}X^\top y$, provided X has **full column rank**. *Worth adding: in practice you never compute the inverse at all — `lstsq` via SVD, never `np.linalg.inv`.*

U5.11 $\sum e_i = 0$; the line passes through $(\bar{x}, \bar{y})$; $\sum x_i e_i = 0$; and SST = SSR + SSE. The first requires an intercept — it *is* the first normal equation.

U5.12 The first normal equation is $\partial\text{SSE}/\partial b_0 = -2\sum(y_i - b_0 - b_1 x_i) = -2\sum e_i = 0$, hence $\sum e_i = 0$ directly.

U5.13 From $\sum e_i = 0$: $\sum(y_i - b_0 - b_1 x_i) = 0$, so $n\bar{y} = nb_0 + b_1 n\bar{x}$, i.e. $\bar{y} = b_0 + b_1\bar{x}$ — which says the point $(\bar{x}, \bar{y})$ satisfies the fitted line.

U5.14 $\sum(y_i - \bar{y})^2 = \sum(\hat{y}_i - \bar{y})^2 + \sum e_i^2$, i.e. SST (total) = SSR (explained by the regression) + SSE (residual). The cross term $2\sum(\hat{y}_i - \bar{y})e_i$ vanishes by the two normal equations.

U5.15 $\text{Var}(b_1) = \sigma^2/S_{xx}$. Therefore **spread the x values out** — put observations at the extremes of the usable range. The slope is estimated more precisely the larger S_{xx} is, and this costs nothing extra per observation.

U5.16 $h_i = 1/n + (x_i - \bar{x})^2/S_{xx}$; the h_i sum to p (2 here). Flag $h_i > 2p/n$. Leverage is about *where the point sits in x*, not about whether its y is unusual.

U5.17 Closed form: $O(nd^2 + d^3)$ once, needs $X^\top X$ or all of X in memory, no tuning, exact answer, fails when d is large or $X^\top X$ is singular. Gradient descent: $O(nd)$ per step over many steps, one mini-batch in memory, requires a step size / schedule / batch size / stopping rule, gets as close as you can afford, almost never fails outright. Closed form extends only to squared loss on a linear model; GD extends to anything differentiable.

U5.18 $y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$ with $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$ **independently and identically**, and x_i fixed and known. Equivalently $y_i \sim \mathcal{N}(\beta_0 + \beta_1 x_i,\ \sigma^2)$.

U5.19 $L = (2\pi\sigma^2)^{-n/2}\exp\!\left(-\frac{1}{2\sigma^2}\sum e_i^2\right)$; $\ell = -\frac{n}{2}\log(2\pi) - \frac{n}{2}\log\sigma^2 - \frac{1}{2\sigma^2}S$, with $S = \sum(y_i - \beta_0 - \beta_1 x_i)^2$.

U5.20 *The sentence to be able to write from memory:* "**Least squares is the maximum likelihood estimator of the linear model under the assumption of independent Gaussian errors with constant variance.**"

U5.21 Because $\ell = A - cS$ where A does not involve β_0, β_1 and $c = 1/(2\sigma^2) > 0$. Maximising $A - cS$ over (β_0, β_1) is minimising S, for *every* positive σ^2 — the value of σ^2 scales the objective but cannot move its argmin.

U5.22 $\hat{\sigma}^2_{\text{ML}} = \text{SSE}/n$; unbiased $s^2 = \text{SSE}/(n-2)$. **Reason:** the residuals satisfy two linear constraints ($\sum e_i = 0$ and $\sum x_i e_i = 0$), so only $n-2$ of them are free; equivalently $E[\text{SSE}] = (n-2)\sigma^2$.

U5.23 $\ell = -n\log(2b) - \frac{1}{b}\sum|e_i|$, so maximising it minimises $\sum|e_i|$ — **least absolute deviations**. Change the noise model and you change the estimator; the recipe, not the squares, is the general fact.

U5.24 $R^2 = 1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}} = \text{SS}_{\text{reg}}/\text{SS}_{\text{tot}}$. On training data with an intercept it lies in $[0, 1]$, because the fit can always fall back on the constant $\bar{y}$. On held-out data it has **no lower bound**: the model was not allowed to see $\bar{y}_{\text{test}}$, so it can do arbitrarily worse than predicting it.

U5.25 Because the larger model *contains* the smaller one — setting the new coefficient to zero reproduces the old fit exactly, so the minimised SS_{res} cannot rise.

U5.26 $R^2_{\text{adj}} = 1 - (1 - R^2)\frac{n-1}{n-p-1}$. It *is* a penalty for parameters that makes nested models roughly comparable. It is **not** a hypothesis test, not a substitute for held-out validation, and it can still be fooled — on 30 rows with 27 noise columns it rose again at $p = 21$ while the cross-validated score collapsed.

U5.27 Two of: it is a training-set number and says nothing about generalisation; it cannot distinguish a good fit from a wrong model, a contaminated dataset or a fit resting on one point (Anscombe); it never falls when predictors are added, so it rewards complexity; and it has no units and no comparison across different y.

U5.28 Linearity — fails, and the *coefficients* are meaningless. Constant variance — fails, and standard errors, t, F, p and all intervals are invalid (the coefficients stay unbiased). Normality — fails, and p-values and intervals are unreliable in small samples. Independence — fails, and standard errors are wrong, usually understated.

U5.29 Fitting the noise rather than the signal: training error falls while error on new data rises. It depends on both because the *same* degree-20 polynomial gave a test RMSE of 53,293 at $n = 15$ and 0.2545 at $n = 1000$. Capacity is only over-capacity relative to the amount of data.

U5.30 (i) The fitted line is unbounded, so it produces "probabilities" outside $[0, 1]$ — a predicted -16.7%. (ii) A single correct but distant point drags the decision boundary, because squared error keeps charging for predictions that are "too right".

U5.31 Odds $= p/(1-p) \in (0, \infty)$; log-odds $= \ln(p/(1-p)) \in (-\infty, \infty)$; $p \in (0, 1)$. Only the **log-odds** has the range of a linear expression, which is exactly why the model

is built there.

U5.32 $\ln\frac{p(x)}{1-p(x)} = w^\top x + b = z$. Inverting: $\frac{p}{1-p} = e^z \Rightarrow p = e^z - pe^z \Rightarrow p(1+e^z) = e^z \Rightarrow p = \frac{e^z}{1+e^z} = \frac{1}{1+e^{-z}} = \sigma(z)$.

U5.33 $\sigma(0) = 1/2$ (so the boundary is the hyperplane $z = 0$); $\sigma(-z) = 1 - \sigma(z)$; $\sigma'(z) = \sigma(z)(1-\sigma(z))$, maximal at $z = 0$ with value 0.25.

U5.34 $\ell = \sum_i [y_i \ln p_i + (1-y_i)\ln(1-p_i)]$. Then $-\ell/n$ is exactly the binary cross-entropy (`log_loss`): maximising the likelihood and minimising cross-entropy are the same operation.

U5.35 $\nabla_w \ell = X^\top(y-p)$, equivalently $\nabla_w L = X^\top(p-y)$ for the negative log-likelihood. In words: how strongly does each feature line up with the errors I am currently making — *prediction minus truth*, projected onto the features.

U5.36 The score equations are $X^\top(\sigma(Xw) - y) = 0$ — the unknown sits inside a transcendental function, like $xe^x = 1$, and cannot be isolated. The normal equations $X^\top Xb = X^\top y$ are *linear* in b and so solve in closed form.

U5.37 $H = -X^\top S X$ with $S = \mathrm{diag}(p_i(1-p_i))$. For $v \neq 0$, $v^\top H v = -\sum_i p_i(1-p_i)(x_i^\top v)^2 < 0$ when X has full column rank, so ℓ is strictly concave. That buys **uniqueness** of the maximum and freedom from local optima — it does **not** buy *existence*: under perfect separation the supremum is approached only as $\|w\| \to \infty$.

U5.38 e^{β_j} is the **multiplicative change in the odds** for a one-unit increase in x_j, holding everything else fixed — and it is the same factor whatever the starting odds. It is *not* a change in the probability.

U5.39 $\left.\frac{\mathrm{d}p}{\mathrm{d}x_j}\right|_{p=0.5} = \beta_j \sigma'(0) = \beta_j/4$. Since $\sigma' \leq 0.25$ everywhere, $|\beta_j|/4$ is an **upper bound** on the change in probability per unit of x_j, attained only at $p = 0.5$.

U5.40 The model outputs a probability; turning it into a decision requires knowing the cost of each kind of error, which is not in the data. The threshold should be chosen by whoever owns those costs — the clinician, the bank, the business — on a *validation* set. Choosing it on the test set is Lecture-7 leakage in a new hat.

U5.41 $P(y = k \mid x) = e^{z_k} / \sum_{m=1}^{K} e^{z_m}$ with $z_k = w_k^\top x + b_k$. For $K = 2$, fix $z_0 = 0$ (legitimate, since adding a constant to every z cancels): $P(y=1) = e^{z_1}/(1+e^{z_1}) = \sigma(z_1)$.

U5.42 A hyperplane separates the classes perfectly. The likelihood increases without bound as $\|w\| \to \infty$ — probabilities are pushed to 0 and 1 — so the MLE **does not exist** (the supremum is not attained). Fix: an L2 penalty, $\min L(w) + \frac{1}{2C}\|w\|^2$, which is scikit-learn's default and the subject of L12.

U5.43 (i) It is non-convex in w when composed with the sigmoid, so the optimisation loses its single-optimum guarantee. (ii) The gradient carries a σ' factor which saturates to zero for badly wrong confident predictions — the model stops learning exactly where it is most wrong. Cross-entropy's $(y-p)$ gradient has no such factor.

§B — Five-mark questions

U5.44 Verified.

Expected answer

(a) $\bar{x} = 2$, $\bar{y} = 4$.

x	y	$x-\bar{x}$	$y-\bar{y}$	$(x-\bar{x})^2$	$(x-\bar{x})(y-\bar{y})$
0	1	−2	−3	4	6
1	2	−1	−2	1	2
2	4	0	0	0	0
3	5	1	1	1	1
4	8	2	4	4	8
				$S_{xx} = \mathbf{10}$	$S_{xy} = \mathbf{17}$

$S_{yy} = 9 + 4 + 0 + 1 + 16 = \mathbf{30}$.
(b) $b_1 = 17/10 = \mathbf{1.7}$; $b_0 = 4 - 1.7(2) = \mathbf{0.6}$; $\hat{y} = 0.6 + 1.7x$.
(c) Fitted **0.6**, **2.3**, **4.0**, **5.7**, **7.4**; residuals **0.4**, **−0.3**, **0.0**, **−0.7**, **0.6**. $\sum e_i = \mathbf{0}$; $\sum x_i e_i = 0(0.4) + 1(-0.3) + 2(0) + 3(-0.7) + 4(0.6) = -0.3 - 2.1 + 2.4 = \mathbf{0}$.
(d) SSE $= 0.16 + 0.09 + 0 + 0.49 + 0.36 = \mathbf{1.10}$; SSR $= \mathbf{28.90}$; SST $= \mathbf{30.00}$; $28.90 + 1.10 = 30.00$ ✓. Checks: SSR $= b_1^2 S_{xx} = 2.89(10) = 28.90$ ✓ and $= S_{xy}^2 / S_{xx} = 289/10 = 28.90$ ✓.
(e) $\hat{y}(6) = 0.6 + 1.7(6) = \mathbf{10.8}$.

U5.45 **Verified.**

Expected answer

(a) $X = \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{pmatrix}$, $X^\top X = \begin{pmatrix} 5 & 10 \\ 10 & 30 \end{pmatrix}$, $X^\top y = \begin{pmatrix} 20 \\ 57 \end{pmatrix}$.
(b) $\det = 5(30) - 10^2 = \mathbf{50}$, and $nS_{xx} = 5(10) = \mathbf{50}$ ✓.
(c) $(X^\top X)^{-1} = \frac{1}{50}\begin{pmatrix} 30 & -10 \\ -10 & 5 \end{pmatrix} = \begin{pmatrix} 0.6 & -0.2 \\ -0.2 & 0.1 \end{pmatrix}$; $b = \frac{1}{50}\begin{pmatrix} 30(20) - 10(57) \\ -10(20) + 5(57) \end{pmatrix} = \frac{1}{50}\begin{pmatrix} 30 \\ 85 \end{pmatrix} = (\mathbf{0.6}, \mathbf{1.7})$ — identical to U5.44.
(d) Fails iff X is rank deficient, i.e. $S_{xx} = 0$, i.e. every x_i equal: e.g. $x = (3, 3, 3, 3, 3)$, giving $\det(X^\top X) = 0$.

U5.46 The six-part derivation. Mark it in six parts, 0.75–1 each:

Expected answer

(1) Object: $\text{SSE}(b_0, b_1) = \sum (y_i - b_0 - b_1 x_i)^2$. (2) $\partial/\partial b_0 = -2\sum(y_i - b_0 - b_1 x_i)$; $\partial/\partial b_1 = -2\sum x_i(y_i - b_0 - b_1 x_i)$. (3) Setting both to zero: $nb_0 + b_1 \sum x_i = \sum y_i$ and $b_0 \sum x_i + b_1 \sum x_i^2 = \sum x_i y_i$. (4) First equation $\Rightarrow b_0 = \bar{y} - b_1 \bar{x}$; substituting into the second gives $b_1(\sum x_i^2 - n\bar{x}^2) = \sum x_i y_i - n\bar{x}\bar{y}$. (5) Recognise $\sum x_i^2 - n\bar{x}^2 = S_{xx}$ and $\sum x_i y_i - n\bar{x}\bar{y} = S_{xy}$, so $b_1 = S_{xy}/S_{xx}$. (6) Second-order: $H = 2\begin{pmatrix} n & \sum x_i \\ \sum x_i & \sum x_i^2 \end{pmatrix}$, $\det H = 4nS_{xx} > 0$ and $2n > 0$, so H is positive definite, SSE is strictly convex, and the stationary point is the unique global minimum.

The expansion identities in (5) have to be shown or quoted, not assumed — however neat the work, an answer that stops after (3) has not finished the question.

U5.47 Verified.

Expected answer

	line	SSE	$\sum e_i$	$\sum x_i e_i$
A	$0.5 + 1.7x$	1.15	+0.5	+1.0
B	$\mathbf{0.6 + 1.7x}$	**1.10**	**0.0**	**0.0**
C	$0.6 + 1.8x$	1.40	−1.0	−3.0
D	$1.0 + 1.5x$	1.50	**0.0**	+2.0

B is the least-squares line: it is the only candidate satisfying *both* normal equations.

Why one is not enough: candidate **D** has $\sum e_i = 0$ exactly — it passes the first test cleanly — yet $\sum x_i e_i = +2.0$ and its SSE is 36% worse than B's. $\sum e_i = 0$ says only that the line passes through $(\bar{x}, \bar{y})$; infinitely many lines do. The second condition is what pins down the slope.

U5.48 Verified.

Expected answer

(a) $g(b_1) = \sum(y_i - b_1 x_i)^2$; $g'(b_1) = -2\sum x_i(y_i - b_1 x_i) = 0 \Rightarrow b_1 \sum x_i^2 = \sum x_i y_i \Rightarrow b_1 = \sum x_i y_i / \sum x_i^2$. $g''(b_1) = 2\sum x_i^2 > 0$, so it is a minimum.
(b) $b_1 = 57/30 = \mathbf{1.9}$.
(c) Residuals **1.0**, **0.1**, **0.2**, **−0.7**, **0.4**; $\sum e_i = \mathbf{+1.0}$, *not* zero.
(d) There is no b_0, so there is no $\partial/\partial b_0$ equation — and $\sum e_i = 0$ *is* that equation. Only $\sum x_i e_i = 0$ survives (check: $0(1.0) + 1(0.1) + 2(0.2) + 3(-0.7) + 4(0.4) = 0$ ✓).
(e) SSE rises from 1.10 to **1.70**. The comparison is fair in the sense that both minimise the same criterion over their own model class, and the no-intercept class is a strict subset — so its SSE can never be smaller. It would be unfair to read the gap as evidence about the data rather than about the constraint.

U5.49 Verified.

Expected answer

(a) $h_i = 0.2 + (x_i - 2)^2/10$: **0.6**, **0.3**, **0.2**, **0.3**, **0.6**; sum = $\mathbf{2.0} = p$ ✓.
(b) $b_1 = \sum(x_i - \bar{x})(y_i - \bar{y})/S_{xx}$. Adding δ to y_i adds δ to y_i and δ/n to $\bar{y}$; the $\bar{y}$ shift contributes $-\frac{\delta}{n}\sum(x_j - \bar{x}) = 0$, leaving $\Delta b_1 = \delta(x_i - \bar{x})/S_{xx}$.
(c) $\delta = 3$, $\Delta b_1 = 3(x_i - 2)/10$:

x_i	Δb_1	new b_1
0	−0.6	1.1
1	−0.3	1.4
2	**0.0**	**1.7**
3	+0.3	2.0
4	+0.6	2.3

(d) The point at $x = 2 = \bar{x}$. Its deviation $(x_i - \bar{x})$ is zero, so it carries no weight in S_{xy} at all — you may move its y anywhere and the slope will not shift by a hair. (The intercept does move.) This is the cleanest demonstration that influence is about position in x, not about having an unusual y.

U5.50 Verified. $\sigma^2 = 4$.

Expected answer

design	S_{xx}	$\mathrm{Var}(b_1)$	$\mathrm{sd}(b_1)$
three at $x = 0$, two at $x = 8$	**76.8**	**0.0521**	**0.2282**
equally spaced 0..8, $n = 5$	40.0	0.1000	0.3162
clustered on $[3, 5]$, $n = 5$	2.5	1.6000	1.2649

Ranking: split-ends best, equally spaced second, clustered worst by a factor of $1.2649/0.2282 = 5.5$ in standard deviation — for the same five measurements. **What the best design cannot do:** with observations at only two x values it cannot detect *curvature*. Two points determine a line, and any non-linearity between them is invisible. Precision on the slope has been bought by giving up the ability to check whether a slope was the right model at all.

U5.51 Mark in five parts:

Expected answer

(1) Model: $y_i \sim \mathcal{N}(\beta_0 + \beta_1 x_i,\ \sigma^2)$ independently. (2) One row: $p(y_i) = (2\pi\sigma^2)^{-1/2} \exp(-e_i^2/(2\sigma^2))$. (3) Independence $\Rightarrow$ product: $L = (2\pi\sigma^2)^{-n/2} \exp(-\frac{1}{2\sigma^2}\sum e_i^2)$. (4) Logs: $\ell = -\frac{n}{2}\log(2\pi) - \frac{n}{2}\log\sigma^2 - \frac{1}{2\sigma^2}S$. (5) Only the last term contains β_0, β_1; it enters with a negative sign and a positive coefficient $1/(2\sigma^2)$. Writing $\ell = A - cS$ with $c > 0$: maximising ℓ is minimising S, *for every* $\sigma^2 > 0$.

U5.52 Verified. SSE $= 1.10$, $n = 5$.

Expected answer

(a) $\hat{\sigma}^2_{\mathrm{ML}} = 1.10/5 = \mathbf{0.22}$. (b) $s^2 = 1.10/3 = \mathbf{0.3667}$; $s = \sqrt{0.3667} = \mathbf{0.6055}$. (c) Bias factor $(n-2)/n = 3/5 = \mathbf{0.6}$; the ML estimate understates σ^2 by $2/n = \mathbf{40}\%$ in expectation. (d) The residuals are not n free numbers: they satisfy $\sum e_i = 0$ and $\sum x_i e_i = 0$, two linear constraints imposed by fitting two parameters. Only $n - 2 = 3$ of the five residuals can be chosen freely, so $E[\mathrm{SSE}] = (n-2)\sigma^2$ and dividing by n systematically under-counts.

U5.53 Verified.

Expected answer

n	SSE	$\hat{\sigma}^2_{\mathrm{ML}}$	s^2	$(n-2)/n$	understatement
8	20	2.5000	3.3333	0.7500	25.00%
6	12	2.0000	3.0000	0.6667	33.33%
12	45	3.7500	4.5000	0.8333	16.67%

Understatement is $2/n$, independent of SSE. Below 1% requires $2/n < 0.01$, i.e. $n > 200$, so $\mathbf{n \geq 201}$.

U5.54 Verified.

Expected answer

(a) $\text{SS}_{\text{tot}} = \mathbf{30.00}$, $\text{SS}_{\text{res}} = \mathbf{1.10}$, $\text{SS}_{\text{reg}} = \mathbf{28.90}$. (b) $R^2 = 1 - 1.10/30.00 = \mathbf{0.9633}$. (c) $\text{SS}_{\text{reg}}/\text{SS}_{\text{tot}} = 28.90/30 = 0.9633$ ✓; $b_1^2 S_{xx}/S_{yy} = 2.89(10)/30 = 0.9633$ ✓. (d) $r = S_{xy}/\sqrt{S_{xx}S_{yy}} = 17/\sqrt{300} = \mathbf{0.9815}$; $r^2 = 0.9633 = R^2$ ✓. (e) It does not tell you whether a *line* was the right model, whether one point is carrying the fit, or whether the variance is constant — Anscombe's four datasets all have $R^2 \approx 0.666$ and four different diagnoses. Look at the **residual plot**.

U5.55 Verified. $n = 25$, $R^2 = 0.55$.

Expected answer

$R^2_{\text{adj}} = 1 - 0.45 \cdot \frac{24}{24-p}$:

p	R^2_{adj}
1	**0.5304**
5	0.4316
10	0.2286
15	−0.2000

Negative when $0.45 \cdot \frac{24}{24-p} > 1$, i.e. $24 - p < 10.8$, i.e. $p > 13.2$: the first integer is $\mathbf{p = 14}$, giving -0.0800 ($p = 13$ still gives $+0.0182$).
Interpretation: once the penalty for parameters exceeds all the variance the model explains, adjusted R^2 goes below the value a constant-only model would score. It is saying that on a per-parameter basis this model has bought nothing — with 14 predictors on 25 rows you are fitting noise, and the un-adjusted R^2 of 0.55 is an illusion of the parameter count.

U5.56 Verified.

Expected answer

(a) $\bar{y}_{\text{test}} = 4$; $\text{SS}_{\text{tot}} = 4 + 0 + 4 = \mathbf{8}$; $\text{SS}_{\text{res}} = 25 + 0 + 25 = \mathbf{50}$; $R^2 = 1 - 50/8 = \mathbf{-5.25}$. (b) Constant 4: $\text{SS}_{\text{res}} = 4+0+4 = 8$, so $R^2 = 1 - 8/8 = \mathbf{0}$. (c) **None** — held-out R^2 is unbounded below. (d) $R^2 \in [0, 1]$ on training data holds because the fitted model *could* have chosen the constant $\bar{y}$ and did at least as well; the constant is inside the model's reach. On held-out data $\bar{y}_{\text{test}}$ is a quantity the model never saw, so there is no guarantee it beats it. A negative held-out R^2 means precisely: you would have done better predicting the mean of the test set for every row.

U5.57 Verified. $x = (1, 2, 3, 4, 5)$, $y = (2, 3, 6, 5, 9)$.

Expected answer

$\bar{x} = 3$, $\bar{y} = 5$; $S_{xx} = \mathbf{10}$, $S_{xy} = \mathbf{16}$, $S_{yy} = \mathbf{30}$. Direct slope $= 16/10 = \mathbf{1.6}$; inverse slope $= S_{yy}/S_{xy} = 30/16 = \mathbf{1.875}$. Ratio $= 1.6/1.875 = \mathbf{0.8533}$; and $r^2 = S_{xy}^2/(S_{xx}S_{yy}) = 256/300 = \mathbf{0.8533}$ ✓. ($r = 0.9238$.)
Which to report: the **direct** line, because the question is to predict y from x and the direct fit is the one that minimises error in y. The two coincide only when $r^2 = 1$.
Inverting the direct fit is wrong: $1/1.6 = 0.625$, but the correct slope for

predicting x from y is $S_{xy}/S_{yy} = 16/30 = 0.533$. The gap is a factor of $1/r^2$. Regression is not symmetric — there is a different line for each direction, and choosing which variable carries the error is part of stating the problem.

U5.58 (a) Faults: reporting a *training* score as evidence of quality; 285 parameters against 331 rows, so the model can very nearly interpolate; no held-out or cross-validated number quoted alongside; and treating a high R^2 as sufficient with no residual diagnostic. (b) A degree-3 expansion of 10 features has $\binom{13}{3} - 1 = 285$ terms against 331 rows — roughly one parameter per row. (c) Its squared error is about **23 times** that of predicting a single constant ($\mathrm{SS}_{\mathrm{res}}/\mathrm{SS}_{\mathrm{tot}} = 1 - (-22.04) = 23.04$); it has learned the training noise and that noise does not recur. (d) The cross-validated score, the held-out score, the parameter count against n, and the trivial baseline for comparison (-0.0001 for the training mean) — with the model rejected in favour of the plain linear fit.

U5.59 Verified.

Expected answer

(a) $\bar{x} = 3.5$, $\bar{y} = 0.5$; $S_{xx} = \mathbf{17.5}$, $S_{xy} = \mathbf{4.5}$; slope $= 4.5/17.5 = \mathbf{0.2571}$; intercept $= 0.5 - 0.2571(3.5) = \mathbf{-0.4}$.
(b) Predictions $\mathbf{-0.1429}$, 0.1143, 0.3714, 0.6286, 0.8857, $\mathbf{1.1429}$ — **two** lie outside $[0, 1]$, one below and one above. Neither is a probability.
(c) $x = (0.5 - b_0)/b_1 = 0.9/0.2571 = \mathbf{3.5}$ mm.
(d) Adding $(20, 1)$: $S_{xx} = \mathbf{250.857}$, $S_{xy} = \mathbf{11.571}$, slope $= \mathbf{0.0461}$, intercept $= \mathbf{0.3013}$, boundary $= \mathbf{4.3086}$ mm — it moved $\mathbf{+0.8086}$ mm. The point at $x = 4$, a genuine malignant case, now predicts $0.4857 < 0.5$ and is **misclassified**.
(e) The added point is already on the correct side and far from the boundary, so it should be irrelevant — but the old line predicts $-0.4 + 0.2571(20) = 4.74$ there, giving a residual of $(4.74 - 1)^2 = 14.0$, larger than the total squared error of all six original points. Squared error keeps charging for predictions that are *too right*, so the fit rotates to reduce that one residual and sacrifices a correct classification to do it.

U5.60 Verified.

Expected answer

p	odds	log-odds
0.05	0.0526	$\mathbf{-2.9444}$
0.20	0.2500	-1.3863
0.50	1.0000	$\mathbf{0.0000}$
0.80	4.0000	$+1.3863$
0.95	19.0000	$\mathbf{+2.9444}$

Antisymmetry: $\mathrm{logit}(1 - p) = -\mathrm{logit}(p)$ — the column is symmetric about zero, and $p = 0.5$ sits exactly at the origin.
Implication: relabelling the classes ($y \mapsto 1 - y$) flips the sign of every coefficient and of the intercept, and leaves the fitted probabilities, the accuracy and the AUC completely unchanged. Which class you call "positive" is a labelling convention, not a modelling decision.

2.5 table, 1 antisymmetry, 1.5 implication.

U5.61 Mark in five parts:

Expected answer

(1) $P(y_i \mid x_i) = p_i^{y_i}(1-p_i)^{1-y_i}$ with $p_i = \sigma(w^\top x_i)$ — the exponents are switches, not arithmetic. (2) $L = \prod_i p_i^{y_i}(1-p_i)^{1-y_i}$. (3) $\ell = \sum_i [y_i \ln p_i + (1-y_i)\ln(1-p_i)]$. (4) Three chain-rule factors: $\frac{\partial \ell}{\partial p_i} = \frac{y_i}{p_i} - \frac{1-y_i}{1-p_i}$; $\frac{\partial p_i}{\partial z_i} = p_i(1-p_i)$; $\frac{\partial z_i}{\partial w_j} = x_{ij}$. (5) **The cancellation:**

$$\left[\tfrac{y_i}{p_i} - \tfrac{1-y_i}{1-p_i}\right] p_i(1-p_i) = y_i(1-p_i) - (1-y_i)p_i = y_i - y_i p_i - p_i + y_i p_i = y_i - p_i.$$

Hence $\partial \ell / \partial w_j = \sum_i (y_i - p_i) x_{ij}$, i.e. $\nabla_w \ell = X^\top (y - p)$.

U5.62 **Verified.**

Expected answer

(a) $z = Xw = (0,0,0,0)$, so $p = (0.5, 0.5, 0.5, 0.5)$. $\ell = 4 \ln 0.5 = \mathbf{-2.772589}$; per-row NLL $= 2.772589/4 = \mathbf{0.693147} = \ln 2$ — the no-information baseline.
(b) $p - y = (+\mathbf{0.5},\ +0.5,\ -0.5,\ -0.5)$. Intercept component: $0.5+0.5-0.5-0.5 = \mathbf{0}$. Feature component: $(-1)(0.5)+0(0.5)+1(-0.5)+2(-0.5) = -0.5-0.5-1.0 = \mathbf{-2.0}$. So $X^\top(p-y) = (\mathbf{0},\ \mathbf{-2.0})$.
(c) $w \leftarrow (0,0) - 0.4(0,-2.0) = (\mathbf{0},\ \mathbf{0.8})$.
(d) $z = (-0.8,\ 0,\ 0.8,\ 1.6)$; $p = (\mathbf{0.310026},\ 0.5,\ \mathbf{0.689974},\ \mathbf{0.832018})$; $\ell = \mathbf{-1.619249}$, an improvement of $+\mathbf{1.153339}$.
(e) The intercept gradient is $\sum_i (p_i - y_i)$. At $w = 0$ every $p_i = 0.5$, and the labels are *balanced* — two zeros and two ones — so the positive and negative discrepancies cancel exactly. The first step therefore adjusts only the slope. With three ones and one zero it would not have been zero.

U5.63 **Verified.** $\beta = \ln 3$.

Expected answer

(a) Odds ratio $= e^{\ln 3} = \mathbf{3}$ — the odds triple.
(b)–(c)

start p	odds	$\times 3$	new p	Δp
0.1	0.1111	0.3333	0.2500	+0.1500
0.2	0.2500	0.7500	0.4286	+0.2286
0.5	1.0000	3.0000	0.7500	+**0.2500**
0.8	4.0000	12.0000	0.9231	+0.1231

Δp is largest starting from $p = 0.5$ and shrinks towards both ends — the same multiplicative change in the odds buys very different changes in probability.
(d) $\beta/4 = 1.0986/4 = \mathbf{0.2747}$, and the largest observed Δp is $0.2500 < 0.2747$ ✓. The bound is not attained because it is the *instantaneous* slope at $p = 0.5$, while a one-unit step is a finite move.
(e) The sentence is wrong twice. $e^\beta = 3$ is a change in the **odds**, not the probability; and it **multiplies** rather than adds, so "by 200%" misdescribes even the odds. Correct: "a one-unit increase multiplies the odds by 3; the effect

on the probability depends on where you start, and is at most 0.2747."

U5.64 Verified.

Expected answer

z	$\sigma(z)$	$1-\sigma(z)$	$\sigma(1-\sigma)$
-3.0	0.0474	0.9526	0.0452
-1.5	0.1824	0.8176	0.1491
0.0	**0.5000**	0.5000	**0.2500**
1.5	0.8176	0.1824	0.1491
3.0	0.9526	0.0474	0.0452

$\sigma(-3) = 0.0474 = 1 - 0.9526 = 1 - \sigma(3)$ ✓, and likewise at ± 1.5. The derivative is largest at $z = 0$, where it equals **0.25**.
Link to divide-by-four: $\mathrm{d}p/\mathrm{d}x_j = \beta_j \sigma'(z)$, and $\sigma' \leq 0.25$ everywhere. So $|\beta_j|/4$ is the maximum possible rate of change of the probability, attained only at the decision boundary $p = 0.5$. The rule is nothing more than this table's middle row.

2.5 table, 1 antisymmetry check, 1.5 the link.

U5.65 Verified.

Expected answer

(a)–(b)

thr	accuracy	precision	recall	F_1	$10\mathrm{FN} + \mathrm{FP}$
0.10	0.8600	0.6610	**0.9750**	0.7879	**30**
0.25	0.9000	0.7551	0.9250	0.8315	42
0.50	**0.9200**	0.8684	0.8250	**0.8462**	75
0.75	0.9133	**0.9655**	0.7000	0.8116	121

(c) Accuracy chooses **0.50**; cost chooses **0.10**. Deploy **0.10**. It has the *worst* accuracy of the four and the lowest cost by a factor of 2.5, because it misses one cancer instead of seven. The F_1 also disagrees with the cost, since F_1 weights precision and recall equally and the problem does not.
(d) ROC–AUC is computed over *all* thresholds — it measures only how well the scores rank positives above negatives. Moving the threshold selects a point on the curve; it does not change the curve. The same model with a different threshold has the same AUC by construction.

U5.66 (a) **Perfect (or quasi-complete) separation**. (b) Once a hyperplane classifies every training row correctly, scaling w up by any factor pushes each p_i closer to its label, so ℓ increases monotonically and its supremum (zero) is approached only as $\|w\| \to \infty$. The supremum is never attained, so there is no maximising w — the MLE does not exist. (c) Accuracy is a function of the *sign* of $w^\top x$, which is unchanged by scaling; it reads 1.0000 at $\|w\| = 2.9$ and at $\|w\| = 51.8$ alike. (d) Add an L2 penalty: $\min L(w) + \frac{1}{2C}\|w\|^2$, which makes the objective coercive and the optimum finite. Cost: the estimates are shrunk and biased, and C becomes a hyperparameter needing cross-validation. Covered in **L12**.

U5.67 Side by side: $X^\top Xb = X^\top y$ (least squares) against $X^\top(\sigma(Xw) - y) = 0$ (logistic). The first is *linear* in b — a $p \times p$ system with a closed-form solution. In the second the unknown sits inside σ, a transcendental function; the equation is of the same character as $xe^x = 1$, which has no solution in elementary functions. Methods used instead: **gradient ascent / descent** (first order, cheap per step, many steps) and **Newton's method / IRLS** (second order, uses $H = -X^\top SX$, quadratic convergence — six steps to machine precision on a small problem). scikit-learn's default is L-BFGS, a quasi-Newton method.

§C — Ten-mark questions

U5.68 Coverage: model and assumptions; squared *vertical* deviations with the differentiability/linear-system argument; the six-part derivation including the Hessian; the matrix form and the rank condition; the four properties; $\mathrm{Var}(b_1) = \sigma^2/S_{xx}$ with the design implication; and the limits — leverage, Anscombe, no causation. Worked fit: U5.44's numbers.

U5.69 Coverage: the probability model; L and ℓ; the $\ell = A - cS$ argument for why σ^2 is irrelevant to the argmin; $\hat{\sigma}^2_{\mathrm{ML}} = \mathrm{SSE}/n$ with the second-derivative check; the bias $(n-2)/n$ with the two-constraints reason; $s^2 = \mathrm{SSE}/(n-2)$; and the Laplace substitution giving LAD. The recipe sentence — state the noise model, write the likelihood, take logs, maximise — should appear.

U5.70 Coverage: SST = SSR + SSE with the vanishing cross term; both definitions of R^2 and the chain $R^2 = \mathrm{SS}_{\mathrm{reg}}/\mathrm{SS}_{\mathrm{tot}} = b_1^2 S_{xx}/S_{yy} = r^2$; monotonicity via the set-the-coefficient-to-zero argument; adjusted R^2 and its failure on 30 rows with 27 noise columns; negative held-out R^2; Anscombe's four datasets with one R^2 and four diagnoses. The sentence to land: **never report an R^2 without a residual plot beside it**.

U5.71 Coverage: definition; why training error cannot detect it (it is monotone in capacity); the degree curve with the turn at degree 3 and the 548× blow-up at degree 20; the n dependence (53,293 at $n = 15$ against 0.2545 at $n = 1000$); CV as the detector, with training picking degree 15 and 5-fold, LOOCV and the test set all picking 3; why one validation split is not enough (ten splits chose degrees $[3, 3, 3, 3, 4, 5, 3, 3, 3, 3]$ with the degree-3 estimate ranging 0.2243–0.4415); and the rule that everything learned is refitted inside every fold.

U5.72 Coverage: the straight-line failure with numbers (U5.59); odds, log-odds, the four-line inversion to σ, and $\sigma' = \sigma(1-\sigma)$; the Bernoulli likelihood and log-likelihood; **the gradient derivation with the cancellation written out**; the transcendental score equations against the linear normal equations; $H = -X^\top SX$ negative definite giving uniqueness but not existence; and one hand-worked gradient step (U5.62). *The cancellation and the uniqueness-not-existence distinction are the two places to be strict.*

U5.73 Coverage: the one-line odds-ratio derivation $e^{z+\beta_j}/e^z = e^{\beta_j}$; the five-starting-probabilities table showing Δp varying by a factor of several while the odds ratio is constant; the divide-by-four rule as an upper bound; why raw coefficients are incomparable across features with different units (and the cm-vs-mm example, $\beta = 1.034$ per mm becoming 10.34 per cm with $e^\beta \approx 3 \times 10^4$); the threshold as a decision made outside the model and on validation data; and a worked cost-weighted threshold

choice (U5.65).

U5.74 A comparison table plus numbers from both. Expect: modelled quantity — $E[y]$ against $\ln \frac{p}{1-p}$; loss — squared error against cross-entropy; closed form — yes against no; objective shape — convex quadratic against strictly concave ℓ; solution — solved against optimised; coefficient — additive change in y against multiplicative change in the odds; pathology — leverage and outlier sensitivity against perfect separation. The sentence worth crediting: *least squares is solved, logistic regression is optimised — which is why Unit III came before Unit V.*

U5.75 Three derivations from one recipe.

Expected answer

Gaussian: $\ell = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum e_i^2 \Rightarrow$ maximise by minimising $\sum e_i^2$ — least squares. **Laplace:** density $\frac{1}{2b}e^{-|e|/b}$ gives $\ell = -n\log(2b) - \frac{1}{b}\sum|e_i| \Rightarrow$ minimise $\sum|e_i|$ — least absolute deviations. **Bernoulli:** $P = p^y(1-p)^{1-y}$ gives $\ell = \sum[y\ln p + (1-y)\ln(1-p)] \Rightarrow$ maximise, i.e. minimise $-\ell/n$ — binary cross-entropy.

§D — Statements

U5.76 **Incomplete.** Differentiability is part of it, but the substantive reason is that the derivative of a square is *linear*, so the first-order conditions form a linear system with a closed-form solution — and squares are what Gaussian maximum likelihood implies. Absolute values are also (almost everywhere) differentiable.

U5.77 **False as stated.** Only when the model has an intercept — $\sum e_i = 0$ *is* the first normal equation. The no-intercept fit of U5.48 has $\sum e_i = +1.0$.

U5.78 **Restates the formula; not an answer.** The reason is that the residuals obey two linear constraints, so only $n-2$ are free and $E[\text{SSE}] = (n-2)\sigma^2$.

U5.79 **Must be qualified.** R^2 measures the proportion of variance in y explained by the model *on the data it was computed from.* It says nothing about generalisation, model correctness, or whether one point is carrying the fit.

U5.80 **False.** Training R^2 can never decrease when a predictor is added, so it cannot arbitrate. Measured: 27 pure-noise columns took R^2 from 0.3738 to 0.9084 on 30 rows.

U5.81 **False** — and it is the error people most often make. e^{β_j} is a **multiplicative** change in the **odds**. The change in probability depends on the starting probability and is bounded by $|\beta_j|/4$.

U5.82 **False.** It is linear in the *log-odds*: $\ln \frac{p}{1-p} = w^\top x + b$. The sigmoid is the inverse link that maps that linear quantity back to a probability; the decision boundary $w^\top x + b = 0$ is a hyperplane.

U5.83 **False, and it is the classic separation tell.** Accuracy depends only on the sign of $w^\top x$ and is invariant to scaling w. The iris example scored 1.0000 with a coefficient of 51.83 and with 2.90 — one of which is a diverging MLE and one of which is a sensible fit.

U5.84 False. `LogisticRegression()` applies L2 regularisation with $C = 1.0$ *by default*. Pass `penalty=None` for the actual MLE — which is also why most users never encounter the separation bug.

Mock paper B — answers

Section A ($5 \times 2 = 10$)

1. As U2.5. The compromise property: quadratic near zero (smooth, like MSE) and linear in the tail (bounded influence, like MAE), joined so that value and slope match.

2. As U3.4: $0 < \eta < 2/L$. At exactly $\eta = 2/L$ the multiplier is $|1 - 2| = 1$: the iterate neither converges nor diverges — it bounces between two points forever. (Measured on $(w-3)^2$ at $\eta = 1$: $0 \to 6 \to 0$.)

3. $(1-p)^d$; $0.95^{20} = \mathbf{0.3585}$.

4. As U4.19 and U4.20: `MinMaxScaler`, because it is defined by $x_{\min}$ and $x_{\max}$, which one bad value sets.

5. As U5.11: $\sum e_i = 0$; the line passes through $(\bar{x}, \bar{y})$; $\sum x_i e_i = 0$; SST = SSR + SSE. The **first** requires an intercept — it is the first normal equation.

Section B (any 3 of 4, 5 each)

6. Identical to **U3.32**. Table, $\eta/\sqrt{1-\gamma} = 1.5811$, s_t falls once $g_t^2 < s_{t-1}$; Adagrad's cannot because it is an undecayed sum of non-negative terms.

7. Identical to **U4.56**(a)–(c) plus the naming of **masking**.

8. Identical to **U5.44**. $S_{xx} = 10$, $S_{xy} = 17$, $b_1 = 1.7$, $b_0 = 0.6$; fitted $0.6, 2.3, 4.0, 5.7, 7.4$; residuals $0.4, -0.3, 0, -0.7, 0.6$; $\sum e_i = \sum x_i e_i = 0$; SSE 1.10, SSR 28.90, SST 30.00, $R^2 = 0.9633$; SSR checked as $b_1^2 S_{xx} = 28.90$.

9. Identical to **U5.62**. $p = (0.5, 0.5, 0.5, 0.5)$, $\ell = -2.772589$, $X^\top(p - y) = (0, -2.0)$, $w = (0, 0.8)$, new $\ell = -1.619249$. The intercept gradient is zero because the labels are balanced and every $p_i = 0.5$, so the discrepancies cancel.

Section C (10)

10. (a) The chain of U3.46, with all seven update rules written correctly — **4 marks**, roughly 0.6 per correct rule with its problem. (b) Curvature and frequency; momentum addresses curvature only — **2 marks**, 1 each, and the second is the one students miss. (c) The derivation of U3.19 and U3.20 — **2 marks**, 1 for $m_t = (1 - \beta_1^t)g$, 1 for the $\pm\eta$ first step. (d) For: robustness to η (2.24 decades against 1.31), works out of the box, handles sparsity. Against: measured *last* of six on diabetes at 30 epochs (0.4929 against momentum's 0.4846), three extra hyperparameters, and the loss got worse without decay. What it buys: a wider window of working learning rates, not a better optimum — **2 marks**, and the last sentence is required for the second of them.

Mock paper C — answers

Section A ($5 \times 2 = 10$)

1. As U1.1. Credit scoring: T = classify an applicant good/bad; E = past applicants with repayment outcomes; P = ROC–AUC, or expected cost under the bank's error costs.

2. As U2.8. At $p = 0.5$ the loss is $-\log 0.5 = \log 2 = \mathbf{0.6931}$.

3. As U3.8 and U3.9: $\eta_{\text{eff}} \approx \eta/(1-\beta)$.

4. $\sqrt{1-p} = \sqrt{0.75} = \mathbf{0.8660}$ — the sd falls to 86.6% of its true value.

5. As U5.31: odds $= p/(1-p) \in (0, \infty)$; log-odds $\in (-\infty, \infty)$; only the **log-odds** matches the range of $w^\top x + b$, which is why the model is built there.

Section B (any 3 of 4, 5 each)

6. **U2.18** and **U2.19** combined: MSE 1.7, MAE 1.0, RMSE 1.3038, Huber at $\delta = 1.5$ is 0.75; shares 0.7353 and 0.5000.

7. Identical to **U4.65**. $S_{xy} = -15$, $S_{xx} = 2$, slope -7.5, intercept 45.8333, $R^2 = 0.0696$ against one-hot's 1.0.

8. **U5.54**(a)–(b) and **U5.52** combined. $R^2 = 1 - 1.10/30 = 0.9633$; $\hat{\sigma}^2_{\text{ML}} = 0.22$; $s^2 = 0.3667$; $s = 0.6055$; bias factor 0.6, understatement 40%. The reason is the two residual constraints, so only $n-2$ are free — *not* "because we divide by n".

9. Identical to **U5.63**. Odds ratio 3; new probabilities $0.25, 0.4286, 0.75, 0.9231$; Δp largest at $p = 0.5$ (+0.25); divide-by-four bound 0.2747; the sentence must be corrected to say the **odds** are **multiplied** by 3.

Section C (10)

10. (a) The rule of U4.37, and five of: a mean or median, a standard deviation, a category vocabulary, a feature subset, a hyperparameter, a resampled row, a PCA basis — **2 marks**. (b) The four leaks with their measured sizes, ranked: resampling +0.78, selection +0.31, scaling +0.0034, imputation +0.0003. The ranking principle: a step that sees y leaks far more than one that does not, and a step that changes the *rows* leaks most of all — **4 marks**, 1 per leak, with the ranking principle required for the fourth. (c) U1.18: selection on all the data planted the correlation it then "found"; honest answer 0.50 — **2 marks**. (d) `Pipeline` (or `ColumnTransformer` inside one) passed to `cross_val_score`/`GridSearchCV`, which refits the whole chain per fold; `imblearn.pipeline.Pipeline` where resampling is involved. The operation it cannot protect you from is **row-dropping** — a transformer may change columns but not rows, so outlier removal and deduplication live outside the pipeline and must be handled by hand — **2 marks**, and the last point is the discriminating one.

End of key. 314 questions, 63 of them numerical. Regenerate every numeric